

Relatorio de Auditoria de Seguranca — Its Time CRM (chat-query)

04 de setembro de 2026 · branch meta · commit 36fa32a

1	3	6	7	1	20
CRITICA	ALTA	MEDIA	BAIXA	INFORMATIVA	PONTOS FORTES

Stack detectada

Camada	Tecnologia identificada
Frontend	React 18 + TypeScript + Vite 7 + Tailwind/shadcn-ui; react-router-dom 7
Backend proprio	Node 22 + Express 5 (TypeScript), ~4.330 linhas de rotas em Project/IA/api-server.ts
Banco / ORM	PostgreSQL gerenciado (Supabase); acesso via supabase-js (PostgREST), sem ORM classico
Auth	Supabase Auth (JWT). Papel e tenant resolvidos em crm.users e espelhados em app_metadata do JWT
Isolamento multi-tenant	Duplo: (a) RLS no Postgres usando current_aces_id()/current_crm_role(); (b) filtro manual por aces_id no backend, que usa service_role
Funcoes serverless	3 Supabase Edge Functions (Deno): buscar-leads, import-leads-csv, send-user-invitation
Deploy	Docker + Docker Swarm (docker-stack.backend.yml), docker-compose.yml para dev, Vercel para o frontend
Migrations	173 arquivos SQL em supabase/migrations/ (schemas crm, agents, collections, calendar, meta, instagram, costs, rb, bi, locator)

Escopo auditado

- Project/IA/ — backend Express + workers + servicos de integracao (167 registros de rota, ~15.900 linhas em sdr-agent-gemini.ts)
- src/ — aplicacao React (paginas, hooks, services, componentes)
- supabase/migrations/ — 173 migrations, com foco em RLS, policies, GRANT/REVOKE e funcoes SECURITY DEFINER
- supabase/functions/ — 3 Edge Functions e supabase/config.toml (verify_jwt)
- docker-compose.yml, docker-stack.backend.yml, Project/IA/Dockerfile, scripts/, docs e arquivos .env*
- Historico do Git (todos os commits alcançaveis) e o bundle compilado em dist/

Nota metodologica

Cada uma das cinco categorias solicitadas foi traduzida para o equivalente desta stack antes da revisao. O mapeamento abaixo descreve como cada categoria foi investigada e o que foi efetivamente percorrido, para que a cobertura da auditoria seja verificavel.

1. Banco sem tranca (isolamento de tenant)

O projeto tem DOIS mecanismos de isolamento e ambos foram mapeados. (a) RLS: foi levantada a lista completa de tabelas com `ENABLE ROW LEVEL SECURITY` nas migrations e cruzada com as 23 tabelas/views que o frontend consulta direto com a chave anon. (b) Filtro manual: o backend usa `SUPABASE_SERVICE_ROLE_KEY` (que ignora RLS), portanto cada handler foi verificado quanto ao filtro explicito por `aces_id`. Views tambem foram checadas quanto a `security_invoker`.

2. Permissao definida no navegador

Cada gate de papel do frontend (`isAdmin`, `isStaff`, `userRole === 'ADMIN'`, `acesId === 535`) foi localizado e rastreado ate a escrita correspondente — seja endpoint do backend, seja policy de RLS — para confirmar se existe verificacao equivalente no servidor.

3. IDOR

Todos os 167 registros de rota de `api-server.ts` foram percorridos (nao amostrados). Para cada handler que delega a uma camada de servico, o metodo de servico foi aberto e classificado por guarda de posse (`ensureAdmin` / `getAgentForAccount` / `loadLeadById` / `ensureInstanceOwnership` / `getAccessibleInstanceNames`). As constraints do banco (FKs compostas (`id`, `aces_id`)) foram lidas para avaliar defesa em profundidade.

4. Chaves expostas

Varredura de arquivos versionados, do historico completo do Git, dos arquivos `.env*` em disco (com checagem de git `check-ignore`), dos `compose/stack` files (com atencao a defaults `${VAR:-valor}`), dos scripts de deploy, da documentacao e do bundle do frontend em `dist/`. Tambem foi verificada a existencia de validacao de startup que rejeite valores default.

5. Inputs sem tratamento (XSS)

Frontend: busca por `dangerouslySetInnerHTML`, `innerHTML/outerHTML/insertAdjacentHTML`, `eval`, `new Function`, `href/src` dinamicos e renderizacao de markdown. Confirmado que nao existe biblioteca de sanitizacao (`DOMPurify/sanitize-html`) no projeto. Backend: procura por respostas HTML e por conteudo do usuario refletido em `res.send()`.

Limites e premissas

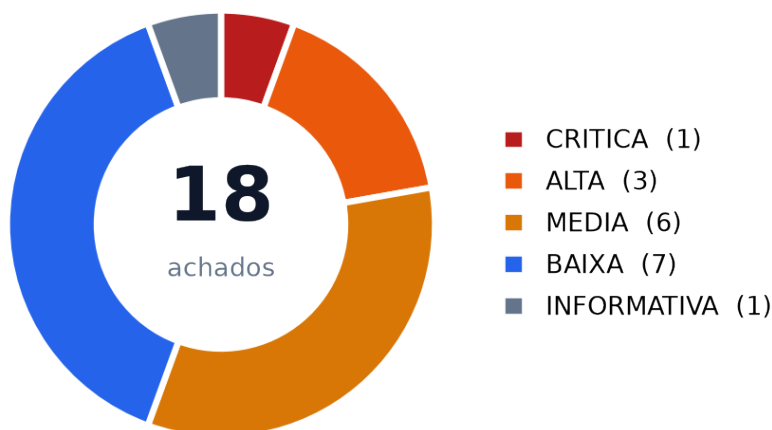
- Nao existe framework de teste de seguranca automatizado no repositorio; a auditoria foi estatica, por leitura de codigo.
- Nao houve acesso ao banco de producao. Afirmacoes sobre RLS valem para o que esta declarado nas migrations.
- Duas condicoes de explorabilidade dependem de variaveis de ambiente nao versionadas (`EVOLUTION_WEBHOOK_SECRET`, `RB_WEBHOOK_JWT_SECRET`) e estao marcadas como tal em cada achado.
- O comportamento de `res.send(string)` do Express 5 (Content-Type: text/html) foi confirmado empiricamente com a versao instalada no projeto.

Resumo executivo

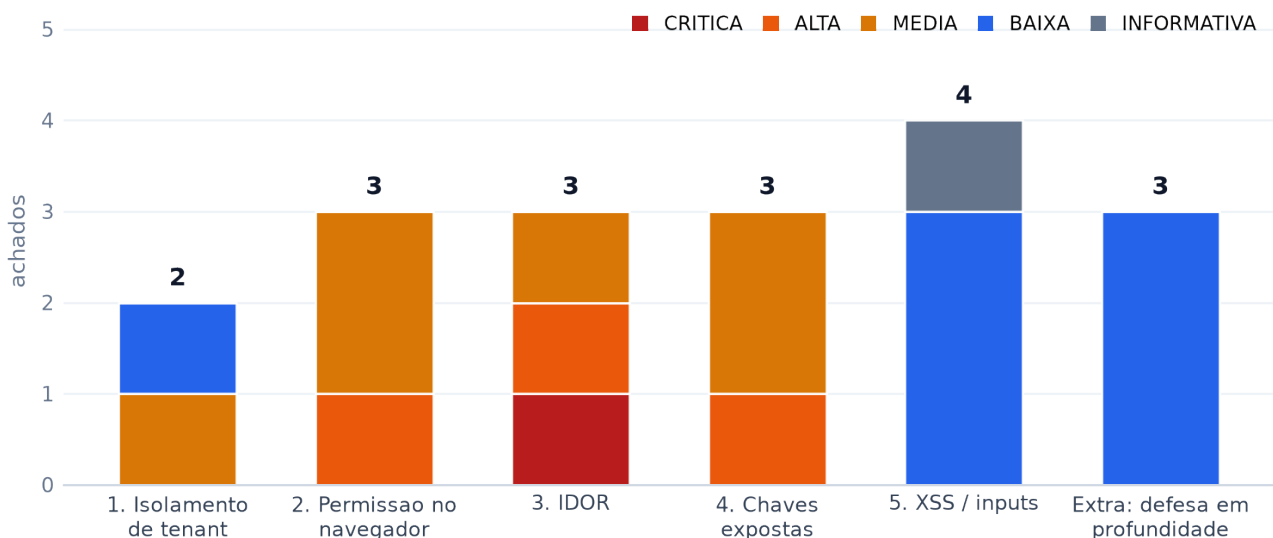
Foram identificados **18 achados** — 1 de severidade critica, 3 alta, 6 media, 7 baixa e 1 informativa — alem de **20 controles verificados e aprovados**. O nucleo do produto se sustenta bem: o isolamento multi-tenant e implementado em duas camadas independentes (RLS no Postgres e filtro manual por aces_id no backend) e a camada de servico do backend apresentou cobertura integral de guardas de posse nos 53 metodos que recebem contexto autenticado.

Os 4 achados de severidade critica e alta estao concentrados nas **bordas de integracao com parceiros** — endpoints que nao passam pelo authMiddleware nem pela camada de servico e que, por isso, ficaram fora do padrao seguido pelo resto do sistema. O caso mais grave e POST /webhook/login, que emite um token de webhook assinado para qualquer tenant sem exigir nenhuma credencial. Somam-se a isso um padrao de *falha aberta* na validacao do webhook da Evolution, a ausencia de autorizacao numa Edge Function de convite de usuario e uma senha de root de producao em texto puro em scripts locais.

Distribuicao por severidade



Distribuicao por categoria (empilhada por severidade)



Veredito por categoria

Categoria	Achados	Pior severidade	Leitura
1. Banco sem tranca (isolamento de tenant)	2	MEDIA	Duplo mecanismo (RLS + filtro por aces_id) implementado com consistencia; falhas pontuais de escopo, sem vazamento sistemico.
2. Permissao definida no navegador	3	ALTA	Maioria dos gates de papel tem equivalente no servidor (RLS ADMIN ou ensureAdmin); 3 excecoes reais.
3. IDOR	3	CRITICA	Objetos por ID sao consistentemente validados na camada de servico; as falhas estao nas rotas que a contornam.
4. Chaves expostas	3	ALTA	Nada sensivel versionado e producao sem defaults; problemas em arquivo local e no historico.
5. Inputs sem tratamento (XSS)	4	BAIXA	Frontend praticamente sem sinks; os achados sao reflexao no backend e endurecimento.
Extra (defesa em profundidade)	3	BAIXA	Itens de robustez que nao se encaixam nas cinco categorias solicitadas.

Pontos fortes verificados

Os itens abaixo foram inspecionados e **aprovados**. Cada um traz a evidencia com arquivo e linha, tanto para registrar o que ja esta protegido quanto para demonstrar a cobertura da auditoria — e, em varios casos, para servir de referencia interna na correcao dos achados.

OK 1. Cobertura integral de guardas de posse na camada de servico do backend

Os 53 metodos publicos de AgentManager que recebem AuthContext foram enumerados e todos aplicam pelo menos uma guarda: ensureAdmin (sdr-agent-gemini.ts:2682), getAgentForAccount (3400), loadLeadById (7847), ensureInstanceOwnership (3317) ou getAccessibleInstanceNames (3342). Nenhum metodo ficou sem guarda. Exemplos: getStageRules (6198), listRuns (6310), resumeLead (6336), sendManualMessage (14956), deleteInstance (7014).

OK 2. Guardas de posse compostas: tenant + instancia + dono do lead

loadLeadById (sdr-agent-gemini.ts:7847-7873) filtra por .eq('aces_id', acesId) e so entao chama canAccessLead. getAgentForAccount (3400-3435) filtra por aces_id e valida created_by ou hasActiveInstanceAccessMembership. ensureInstanceOwnership (3317-3340) confere aces_id da instancia e a associacao ativa do usuario. O modelo nao para no tenant: desce a instancia e ao dono do registro.

OK 3. FKs compostas (id, aces_id) impedem referencia cross-tenant no schema collections

20260903205231_collections_core_v1.sql declara constraints como agent_source_bindings_tool_tenant_fkey FOREIGN KEY (agent_tool_id, aces_id) REFERENCES agents.agent_tools(id, aces_id) (linhas 115-120), journey_rules_funnel_tenant_fkey (322-325), mapping_profiles_connection_tenant_fkey (368-371) e spreadsheet_imports_connection_tenant_fkey (404-407). Isso torna a violacao de tenant impossivel no banco, nao apenas improvavel no codigo — e foi o que neutralizou o risco de leitura em F06.

OK 4. Schema collections fechado para a chave anon e com RLS por padrao

20260903205231_collections_core_v1.sql:492-513 percorre as 14 tabelas do schema executando ENABLE ROW LEVEL SECURITY, REVOKE ALL ... FROM PUBLIC, anon, authenticated e concedendo apenas ao service_role. As 24 rotas /api/collections/* passam todas por requireCollectionAdmin (api-server.ts:1022-1028) e propagam context.acesId em cada consulta e RPC.

OK 5. Webhook de ingestao de cobranca com autenticacao completa

collections/webhook-security.ts:28-55 valida HMAC-SHA256 sobre timestamp + '.' + corpo cru, exige timestamp na janela de tolerancia (anti-replay) e compara com timingSafeEqual, aceitando segredo atual e anterior para rotacao sem downtime. collection-api.ts:54-142 soma rate limit por IP via RPC consume_webhook_rate_limit, Idempotency-Key obrigatorio, express.raw com limite de 1 MB e mensagens de erro genericas que nao distinguem fonte inexistente de assinatura invalida.

OK 6. Webhooks Meta e Instagram falham fechados, com comparacao em tempo constante

meta-webhook.ts:56-70 e instagram-webhook.ts:110-120 retornam false quando appSecret esta ausente e usam crypto.timingSafeEqual apos checagem de tamanho. Os handlers em api-server.ts:657-670 e 769-782 lancam 401 antes de qualquer processamento. Este e o comportamento correto que F02 nao replica.

OK 7. Entrega de midia do Instagram por token de capacidade assinado

instagram-service.ts:158-171 gera um token HMAC-SHA256 sobre {acesId, attachmentId, expiresAt}; downloadMediaDelivery (173-213) valida a assinatura com timingSafeEqual, rejeita expirados e TTL inflado, e busca o intent com .eq('aces_id', acesId).eq('attachment_id', attachmentId). O MIME e restrito por allowlist na ingestao (instagram-workers.ts:85) e a resposta traz X-Content-Type-Options: nosniff (api-server.ts:815) — por isso o endpoint publico da linha 807 nao virou achado.

OK 8. Chat interno com client escopado por RLS mais checagem explicita de participacao

internal-chat-service.ts:105-111 cria client com a chave anon e o JWT do usuario, de modo que a RLS se aplica; assertMember (113-121) confirma participacao ativa antes de listMessages (287) e createAttachmentUploadUrl (530); o envio passa pelo RPC rpc_send_internal_message. Intents de anexo sao validados por .eq('aces_id').eq('created_by') e conferencia de existencia e tamanho real do objeto no Storage (443-511).

OK 9. RLS com escrita restrita a ADMIN nas tabelas de configuracao

crm.pipelines (20260706231251:27-65), crm.pipeline_stages (20260410100000:962-988), crm.automation_funnels/steps/executions (20260411190000:84-200), crm.empresas (20260724202355:251-280) e as 7 tabelas do schema calendar (20260724202355:844-899) exigem `current_crm_role() = 'ADMIN'` ou `crm.current_user_is_account_admin()` em INSERT/UPDATE/DELETE. E o que sustenta no servidor os gates isAdmin de KanbanColumn.tsx:117, PipelineToolbar.tsx:106 e Dashboard.tsx:109.

OK 10. Claims de tenant e papel em app_metadata, nao editaveis pelo usuario

20260411161000_move_crm_context_to_jwt.sql:23-51 define `current_crm_user_id`, `current_acces_id` e `current_crm_role` lendo `auth.jwt()` -> 'app_metadata'. `app_metadata` so e gravavel pelo `service_role` (trigger `sync_acces_id_to_jwt`, linhas 3-21) — diferente de `user_metadata`, que o proprio usuario poderia alterar via `auth.updateUser` e que resultaria em auto-promocao a ADMIN. A escolha esta certa.

OK 11. RLS de leads desce ao nivel de instancia e de dono do registro

20260716195449_harden_tenant_instance_rls.sql:353-381 substitui a comparacao simples de `aces_id` por `crm.current_user_can_access_lead(id)` no SELECT, `crm.current_user_can_edit_lead(id)` no UPDATE/DELETE e, no INSERT, exige `aces_id + current_user_can_access_instance(instancia,'editor') + crm_user_belongs_to_current_account(owner_id)`. `crm.message_history` (386-393) herda o mesmo modelo.

OK 12. Views expostas ao frontend com security_invoker habilitado

`crm.v_lead_details` recebe `ALTER VIEW ... SET (security_invoker = true)` apos cada redefinicao (20260425110000:229, 20260618205533:55, 20260708211239:133, 20260724202355:593, 20260827145250:151), de modo que a RLS de `crm.leads` continua valendo atraves da view. As views de `costs` (20260716195449:362) e de `superadmin` (20260814001041:442, 479, 501) usam `WITH (security_invoker = true)` na criacao. A execucao e `vw_relatorio_leads`, que nao esta versionada (F08).

OK 13. Rotas de superadmin isoladas por RPC de staff e prefixo de RPC restrito

As 21 rotas `/api/admin/(overview|accounts|plans|revenue-entries|fixed-costs|exchange-rates)` usam `authMiddleware + requireStaff` (`api-server.ts:864-871`), que consulta o RPC `service_admin_is_staff` (`sdr-agent-gemini.ts:10353-10359`). `adminRpc` (10361-10368) recusa qualquer nome de funcao que nao comece com `'service_admin_'`, bloqueando execucao arbitraria de RPC mesmo se o nome vazasse para o corpo da requisicao.

OK 14. Edge Functions com verify_jwt=false compensam com verificacao interna

`buscar-leads` e `import-leads-csv` estao com `verify_jwt = false` em `supabase/config.toml:62` e `68`, mas ambas exigem o header `Authorization`, resolvem o usuario com `userClient.auth.getUser()` usando a chave `anon`, carregam `crm.users` e escopam todas as consultas por `crmUser.acces_id` (`buscar-leads/index.ts:242-276`, `342-350`, `463-473`; `import-leads-csv/index.ts:56-91`, `205-244`, `275-308`). Esse e exatamente o padrao ausente em `send-user-invitation` (F03).

OK 15. Gestao de segredos: nada versionado, sem defaults em producao, bundle limpo

`docker-stack.backend.yml` usa `${VAR}` puro nos 18 segredos (linhas 6-9, 28, 33-40, 47-51, 70-72), sem nenhum default. Todos os `.env*` (`.env.local`, `.env.vps.local`, `.env.cloudflare`, `.env.vercel`, `Project/IA/.env.local`, `DEPLOY_ENV_VPS.local.md`) confirmados como ignorados por `git check-ignore`. A varredura de todo o 'git ls-files' por `AlzaSy*`, `sk-*`, `JWT HS256` e blocos `PRIVATE KEY` voltou vazia, e `dist/assets/*.js` nao contem `service_role` nem chave de API. `scripts/set-backend-secret.ps1` le o segredo por `Read-Host -AsSecureString` e nunca o imprime.

OK 16. URLs de anexo sempre assinadas pelo servidor, sem esquema controlado pelo usuario

`src/components/chat/MessageAttachment.tsx:205` e `240` usam `href={attachment.url}`, e esse valor vem sempre de `createSignedDownloadUrl` (`sdr-agent-gemini.ts:7456-7466`), chamado nos 5 pontos que montam anexos (9422, 9797, 14315, 14850, 15675). O resultado e sempre uma URL `https` do Supabase Storage, logo nao existe caminho para javascript: em `href`.

OK 17. Frontend sem os sinks classicos de XSS

Varredura em todo o `src/` nao encontrou `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval` nem `new Function`. Nao ha renderizador de markdown/HTML no `package.json`. Os unicos dois `dangerouslySetInnerHTML` sao blocos `<style>` (F18), sendo um deles CSS estatico. Conteudo de lead, mensagem e nota e renderizado como texto em JSX, com escape automatico do React.

OK 18. CORS do backend por allowlist estrita

`api-server.ts:583-595` monta a allowlist de `CORS_ORIGINS` e so envia `Access-Control-Allow-Origin` quando a origem consta da lista, acrescentando `Vary: Origin`. Nao ha reflexao incondicional de origem nem curinga com credenciais.

OK 19. Isolamento de tenant nas rotas administrativas de conta

As rotas de gestao de usuarios, acessos e empresas verificam papel e escopo em toda consulta: `/api/admin/users` (2063-2085), `/api/admin/users/:id/role` (2087-2160, com `.eq('id').eq('aces_id')` e revogacao em cascata de `instance_access_memberships` e `empresa_memberships`), `/api/admin/instance-access` (2163-2277, validando instancia e vendedor na conta antes do upsert), `/api/admin/company-access` (2570-2670) e `/api/admin/companies/:id` (2486-2524).

OK 20. RPCs de servico validam o tenant dentro da funcao

`crm.service_move_lead_to_stage` (20260411210000:269-316) e `SECURITY DEFINER` com `SET search_path`, confere a etapa com `WHERE id = p_stage_id AND aces_id = p_aces_id` e levanta excecao quando nao encontra, repetindo a checagem no `UPDATE` do lead. As permissoes sao concedidas somente ao `service_role`, com `REVOKE` explicito para `authenticated`, `anon` e `PUBLIC`.

Pontos fracos — riscos centrais

Os seis temas abaixo sintetizam a causa-raiz dos achados. Corrigir apenas os sintomas resolve esta auditoria; tratar estes temas evita a reincidencia.

R1 A borda de integracao e o elo fraco, nao o nucleo multi-tenant

O nucleo do produto — rotas autenticadas, RLS, camada de servico — esta consistente. Os achados de maior severidade estao todos nas bordas de integracao com parceiros: /webhook/login sem credencial (F01), webhook da Evolution falhando aberto (F02) e a Edge Function de convite sem autorizacao (F03). Sao caminhos que nao passam pelo authMiddleware nem pela camada de servico e, por isso, escaparam do padrao que o resto do sistema segue.

R2 Falha aberta como default em vez de falha fechada

Duas verificacoes de credencial tratam 'segredo nao configurado' como 'autorizado': validateWebhookSecret da Evolution (F02) e, parcialmente, o handler da Gupshup — que ao menos barra producao. O padrao correto ja existe no proprio repositorio (Meta, Instagram e collections falham fechados). O risco e de configuracao: um ambiente sem a variavel fica aberto sem qualquer sinal.

R3 Ausencia de validacao de startup para qualidade de segredo

requireEnv (api-server.ts:274-280) apenas verifica se a string e nao vazia. Nada impede que 'local-only-rb-webhook-secret' (docker-compose.yml:25), 'local-dev-verify-token' ou 'local-dev-app-secret' (.env.local.example:57-58) cheguem a um ambiente com dados reais. O sistema sobe normalmente e assina tokens com um segredo publicado no repositorio (F09).

R4 Client service_role sem RLS torna o filtro manual a unica barreira

O backend usa SUPABASE_SERVICE_ROLE_KEY em praticamente todas as consultas, o que anula a RLS. Onde o filtro por aces_id existe, a protecao e solida; onde falta, o vazamento e imediato — foi assim em F07 (requireChannel sem acesId) e F06 (upsert por PK). Nao ha lint, wrapper tipado nem teste que force a presenca do filtro, o que deixa a garantia dependente de revisao humana.

R5 Objetos de banco fora do versionamento

vw_relatorio_leads e consultada pelo frontend com a chave anon (src/pages/Leads.tsx:119) e nao existe em nenhuma das 173 migrations. Sua postura de RLS e portanto nao auditavel a partir do repositorio, e como views nascem com security_invoker = false o cenario padrao e o inseguro (F08). O gate de acesso correspondente e um if no navegador.

R6 Credencial de producao em texto puro protegida apenas por glob de .gitignore

A senha de root do VPS aparece literalmente em tres scripts (F04). O que evita o commit e a regra /scripts/*vps*.js do .gitignore — renomear o arquivo, copiar o trecho ou usar git add -f derrama a credencial no historico. Nao existe hook de pre-commit nem varredura de segredos na CI.

Indice de achados

Visao consolidada. O detalhamento por categoria vem nas paginas seguintes.

Sev.	ID	Achado	Arquivo:linha principal
CRITICA	F01	Emissao de token de webhook sem nenhuma autenticacao (/webhook/login)	Project/IA/api-server.ts:841-853
ALTA	F02	Validacao do webhook da Evolution falha aberta quando o segredo nao esta configurado	Project/IA/sdr-agent-gemini.ts:15828-15835
ALTA	F03	Edge Function send-user-invitation sem autorizacao e sem vinculo convite<->e-mail	supabase/functions/send-user-invitation/index.ts:38-41
ALTA	F04	Senha de root do VPS de producao em texto puro em scripts do projeto	scripts/get_vps_logs.js:36-39
MEDIA	F05	POST /api/agents/:id/tools/rb_billing/run-now sem verificacao de papel no servidor	Project/IA/api-server.ts:2928-2937
MEDIA	F06	Upsert por id fornecido pelo cliente permite sequestro de linhas de outro tenant	Project/IA/api-server.ts:1345-1362
MEDIA	F07	Sincronizacao de templates Meta nao filtra por tenant	Project/IA/api-server.ts:1488-1508
MEDIA	F08	Export de relatorio de leads restrito apenas no navegador, sobre view sem RLS rastreada	src/pages/Leads.tsx:112-137
MEDIA	F09	Segredo default publico no docker-compose e ausencia de validacao de startup	docker-compose.yml:25
MEDIA	F10	Segredos de terceiros commitados no historico do Git	referencias/Cobranca API/Curl.txt (commit 02333eb e seguintes)
BAIXA	F11	XSS refletido no challenge de verificacao dos webhooks Meta e Instagram	Project/IA/api-server.ts:757-767
BAIXA	F12	Open redirect por URL protocolo-relativa no returnPath do OAuth do Instagram	Project/IA/instagram-service.ts:89-92
BAIXA	F13	Erros crus do banco devolvidos ao cliente no campo details	Project/IA/api-server.ts:4315-4321
BAIXA	F14	Limite default de corpo JSON em 150 MB	Project/IA/api-server.ts:568-580
BAIXA	F15	Comparacao de segredos de webhook fora de tempo constante	Project/IA/sdr-agent-gemini.ts:15834
BAIXA	F16	Sessao Supabase persistida em localStorage	src/integrations/supabase/client.ts:15-24
BAIXA	F17	Claims de papel e tenant no JWT ficam obsoletas ate a renovacao do token	supabase/migrations/20260411161000_move_crm_context_to_jwt.sql:3-51
INFORMATIVA	F18	Sink de CSS via dangerouslySetInnerHTML em componente de chart (latente)	src/components/ui/chart.tsx:61-88

Achados detalhados por categoria

Para cada achado: severidade, arquivo e linha exatos, trecho do código, por que e explorável, impacto, condicao de explorabilidade, correcao sugerida e critérios de aceite verificaveis.

1. Banco sem tranca (isolamento de tenant)

2 achado(s)

MEDIA

F07 — Sincronizacao de templates Meta nao filtra por tenant

Localizacao

Project/IA/api-server.ts:1488-1508

Project/IA/meta-template-service.ts:106-118

Project/IA/meta-template-service.ts:56-97

Trecho do código

```
// api-server.ts:1488 - instanceName vem do corpo e nao e validado contra o tenant
const result = await metaTemplateService.syncTemplatesForInstance(instanceName);

// meta-template-service.ts:106 - requireChannel sem acesId
private async requireChannel(instanceName: string) {
  const { data, error } = await this.metaClient
    .from("whatsapp_channels")
    .select("id, waba_id, access_token_secret_ref")
    .eq("instance_name", instanceName) // <-- falta .eq("aces_id", acesId)
    .maybeSingle();
```

Por que e explorável

O handler confere apenas que o chamador e ADMIN e repassa instanceName cru. requireChannel busca o canal so por instance_name usando client service_role, sem escopo de conta. Todas as rotas irmas fazem o certo: metaAdminService.upsertChannel passa por requireInstance(acesId, instanceName) (meta-admin-service.ts:182-189) e listTemplates recebe acesId. Nomes de instancia sao curtos e previsiveis (o script verify-evolution-env.mjs cita 'prospect', 'Juan', 'Lucas'), o que torna a adivinhacao viavel.

Impacto

Escrita cross-tenant: um ADMIN do tenant A sobrescreve meta.whatsapp_templates e last_template_sync_at do canal do tenant B e provoca chamadas a Graph API da Meta usando o access token do tenant B (consumo de quota e ruido nos logs do parceiro). A resposta devolve apenas { instanceName, mode, synced }, portanto nao ha leitura direta do conteudo alheio.

Condicao de explorabilidade

Requer papel ADMIN em algum tenant e conhecer o nome de instancia da vitima.

Correcao sugerida

Propagar o tenant: mudar a assinatura para syncTemplatesForInstance(acesId, instanceName) e adicionar .eq('aces_id', acesId) em requireChannel; ou validar antes no handler com metaAdminService.requireInstance(context.acesId, instanceName). Vale varrer meta-template-service.ts em busca de outras consultas sem escopo de conta.

Craterios de aceite

- [] syncTemplatesForInstance recebe e aplica acesId.
- [] requireChannel filtra por aces_id e instance_name.
- [] Chamada com instanceName de outro tenant retorna 404 e nao altera nenhuma linha.
- [] Teste com dois tenants confirmando isolamento.

BAIXA

F17 — Claims de papel e tenant no JWT ficam obsoletas ate a renovacao do token**Localizacao****supabase/migrations/20260411161000_move_crm_context_to_jwt.sql:3-51****src/contexts/AuthContext.tsx:76-92****Project/IA/sdr-agent-gemini.ts:2647-2679****Trecho do codigo**

```
-- 20260411161000:43 — RLS le o papel do app_metadata do JWT
CREATE OR REPLACE FUNCTION public.current_crm_role()
RETURNS crm.user_role LANGUAGE sql STABLE SECURITY DEFINER
AS $$ SELECT NULLIF(auth.jwt() -> 'app_metadata' -> 'crm_role', '')::crm.user_role; $$;

-- o trigger sync_aces_id_to_jwt atualiza auth.users.raw_app_meta_data,
-- porem o JWT ja emitido continua valido com o papel antigo
```

Por que e explorável

A RLS decide papel e tenant a partir das claims do JWT, gravadas em app_metadata pelo trigger sync_aces_id_to_jwt. app_metadata nao e editavel pelo usuario (so pelo service_role), portanto nao ha spoofing — esse ponto esta correto. O efeito colateral e temporal: rebaixar um usuario de ADMIN para VENDEDOR via PATCH /api/admin/users/:id/role atualiza a tabela e o app_metadata, mas o access token em uso continua carregando crm_role = ADMIN ate expirar (1h por default no Supabase). Durante essa janela, escritas diretas do frontend pela chave anon ainda passam pelas policies de ADMIN. O backend nao sofre disso porque authenticate() (sdr-agent-gemini.ts:2647) le o papel ao vivo de crm.users. AuthContext.tsx:76-92 detecta a divergencia e chama refreshSession(), o que mitiga o caso comum, mas e logica de cliente e pode ser evitada.

Impacto

Janela de privilegio residual de ate uma expiracao de token depois de um rebaixamento de papel ou troca de tenant, aplicavel somente as escritas que o frontend faz direto no Postgres (automation_*, pipelines, pipeline_stages, empresas, calendar.*).

Condicao de explorabilidade

Janela limitada ao tempo de vida do access token e parcialmente mitigada pelo refresh automatico do AuthContext.

Correcao sugerida

Nas policies mais sensiveis, resolver o papel na tabela em vez da claim (por exemplo, uma funcao SECURITY DEFINER que faca SELECT role FROM crm.users WHERE auth_user_id = auth.uid()), aceitando o custo de um lookup. Alternativamente, invalidar as sessoes do usuario no servidor ao alterar papel (auth.admin.signOut do usuario) e reduzir o tempo de vida do access token.

Critérios de aceite

- [] Rebaixar um usuario de ADMIN para VENDEDOR revoga imediatamente as escritas admin feitas direto pelo frontend.
- [] Teste de integracao: alterar papel e, sem renovar o token, confirmar que a escrita admin e negada.
- [] Decisao documentada entre lookup na tabela e invalidacao de sessao.

2. Permissao definida no navegador**3 achado(s)****ALTA****F03 — Edge Function send-user-invitation sem autorizacao e sem vinculo convite<->e-mail****Localizacao****supabase/functions/send-user-invitation/index.ts:38-41****supabase/functions/send-user-invitation/index.ts:43-55****supabase/functions/send-user-invitation/index.ts:74-77****supabase/config.toml:49-51****Trecho do codigo**

```
const { email, invitationId } = body ?? {};  
if (!email || !invitationId) { /* ... 400 ... */ }  
  
const supabaseAdmin = createClient(  
  Deno.env.get("SUPABASE_URL") ?? "",  
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? "", // service_role: ignora RLS  
);  
  
const { data: invitation } = await supabaseAdmin  
  .schema("crm").from("user_invitations")  
  .select("*").eq("id", invitationId).single(); // sem filtro por aces_id  
  
await supabaseAdmin.auth.admin.inviteUserByEmail(email, { // email vem do atacante  
  data: { name: invitation.name, role: invitation.role,  
    aces_id: invitation.aces_id, invitation_id: invitationId },  
});
```

Por que e explorável

A funcao nao chama `auth.getUser()`, nao verifica papel e nao compara o parametro email com `invitation.email`. O convite e buscado com `service_role` apenas por id, sem filtro por tenant. `verify_jwt = true` (config.toml:51) garante somente que exista um JWT valido do projeto — qualquer usuario autenticado serve, de qualquer papel e de qualquer tenant. A policy `inv_select` (20260411154500_fix_crm_rls_recursion.sql:105-107) permite a qualquer usuario autenticado ler os convites do seu proprio `aces_id`, incluindo os de role ADMIN, o que fornece um `invitationId` valido.

Impacto

Escalonamento de privilegio: um VENDEDOR ou NENHUM le um `invitationId` com role ADMIN da sua conta e chama a funcao com um e-mail que controla; o Supabase envia convite carregando `role=ADMIN` e `aces_id` da conta no `app_metadata` do novo usuario, resultando em conta ADMIN sob controle do atacante. O parametro email desacoplado do convite tambem permite redirecionar qualquer convite pendente para um endereco arbitrario.

Condicao de explorabilidade

Requer apenas uma sessao autenticada qualquer no projeto Supabase e um `invitationId` valido.

Correcao sugerida

Na propria funcao: (1) resolver o chamador com um client anon usando o header `Authorization` e `auth.getUser()`; (2) carregar o `crm.users` do chamador e exigir role ADMIN; (3) filtrar o convite por `aces_id` do chamador; (4) ignorar o email do corpo e usar exclusivamente `invitation.email`. Os arquivos `buscar-leads/index.ts:56-91` e `import-leads-csv/index.ts:242-276` ja implementam exatamente esse padrao e servem de referencia.

Critérios de aceite

[] A funcao resolve o chamador via `auth.getUser()` e rejeita com 401 quando nao ha usuario valido.

[] A funcao rejeita com 403 quando o chamador nao tem role ADMIN.

- [] O convite e carregado com filtro .eq('aces_id', callerAcesId); id de outro tenant retorna 404.
- [] O destinatario do convite vem de invitation.email; o campo email do corpo e ignorado ou validado como igual.
- [] Teste cobrindo: sem JWT -> 401; JWT de VENDEDOR -> 403; invitationId de outro tenant -> 404; email divergente -> rejeitado.

MEDIA**F05 — POST /api/agents/:id/tools/rb_billing/run-now sem verificacao de papel no servidor**

Localizacao

Project/IA/api-server.ts:2928-2937

Project/IA/rb-billing-worker.ts:459-462

Project/IA/sdr-agent-gemini.ts:5145-5147

src/services/agentToolsService.ts:380

Trecho do codigo

```
// api-server.ts:2928 — nao chama manager.*, portanto nao passa por ensureAdmin
app.post(
  "/api/agents/:id/tools/rb_billing/run-now",
  authMiddleware,
  asyncHandler(async (req: AuthenticatedRequest, res) => {
    const agentId = getSingleParam(req.params.id);
    const result = await rbBillingWorker.runNowForAgent(
      req.authContext!.acesId, agentId); // so escopo de tenant, sem papel
    res.status(202).json({ success: true, result });
  }),
);

// contraste — a rota irma valida papel:
// sdr-agent-gemini.ts:5145 async bootstrapRbBilling(...) { this.ensureAdmin(context); ... }
```

Por que e explorável

Essa e a unica rota de /api/agents/:id/tools/* que nao delega ao manager (que sempre chama ensureAdmin + getAgentForAccount). Ela chama rbBillingWorker.runNowForAgent diretamente, e o getBindingForAgent (rb-billing-worker.ts:479-486) filtra apenas por aces_id e agent_id. O isolamento de tenant esta correto; o que falta e o controle de papel. No frontend, toda a pagina de Agentes esta atras de isAdmin (src/components/layout/Sidebar.tsx:202), logo a restricao existe so no navegador.

Impacto

Qualquer usuario autenticado do tenant, inclusive com role NENHUM ou VENDEDOR, dispara uma rodada de cobranca fora de hora: envio de mensagens de cobranca a leads reais via WhatsApp, consumo de orcamento de IA e poluicao dos registros de execucao. Nao ha exposicao de dados de outro tenant.

Condicao de explorabilidade

Requer sessao autenticada no tenant e um agente com a tool rb_billing configurada.

Correcao sugerida

Roteá-la pelo manager, como as demais: adicionar uma verificacao explicita de papel antes da chamada (const context = req.authContext!; if (context.role !== 'ADMIN') throw new HttpError(403, ...)) e validar a posse do agente com manager.getAgentForAccount(agentId, context.acesId, context.crmUserId, context.role) — de preferencia expondo um metodo runRbBillingNow(context, agentId) no AgentManager para manter o padrao.

Critérios de aceite

- [] Chamada com token de usuario VENDEDOR/NENHUM retorna 403.
- [] Chamada com agentId de outro tenant retorna 404.
- [] A rota valida posse do agente pela mesma funcao usada pelas rotas irmas.

[] Teste de rota cobrindo os tres casos (NENHUM -> 403, VENDEDOR -> 403, ADMIN -> 202).

MEDIA**F08 — Export de relatorio de leads restrito apenas no navegador, sobre view sem RLS rastreada**

Localizacao

src/pages/Leads.tsx:112-137

src/integrations/supabase/client.ts:15-24

Trecho do codigo

```
// src/pages/Leads.tsx:112
const { profile } = await getCrMBackend<CrMProfileResponse>("/api/crm/profile");
const acesId = profile?.aces_id;

// 3. Verificar se e o aces_id especial (535)
if (acesId === 535) { // <-- gate 100% client-side
  let query = supabase
    .from('vw_relatorio_leads') // view nao definida em supabase/migrations/
    .select('*');
  ...
  const { data: viewData } = await query;
  exportGenericToCSV(viewData, ...);
}
```

Por que e explorável

A decisao de acesso ao relatorio esta num if do bundle React: qualquer usuario autenticado pode chamar `supabase.from('vw_relatorio_leads').select('*')` pelo console do navegador com a chave anon, sem passar por esse if. O que impede o vazamento e exclusivamente a postura de RLS da view — e `vw_relatorio_leads` NAO existe em nenhuma das 173 migrations (foi criada fora do versionamento). Views no Postgres nascem com `security_invoker = false`, isto e, consultam as tabelas-base como o owner e ignoram a RLS de `crm.leads`. Todas as outras views do projeto tratam isso explicitamente (`crm.v_lead_details` recebe `ALTER VIEW ... SET (security_invoker = true)` apos cada redefinicao — 20260618205533:55, 20260708211239:133, 20260724202355:593, 20260827145250:151).

Impacto

Se a view em producao estiver com `security_invoker = false` e `SELECT` concedido a `authenticated`, qualquer usuario logado de qualquer tenant extrai a base de leads inteira de todas as contas (dados pessoais: nome, telefone, e-mail, cidade, historico de etapa) — vazamento de LGPD em massa. Nao foi possivel confirmar o estado real em producao porque a view nao esta versionada; por isso o achado esta em media e nao em critica.

Condicao de explorabilidade

Explorabilidade depende da configuracao real da view em producao (`security_invoker` e `GRANT`), nao verificavel a partir do repositorio.

Correcao sugerida

Primeiro, verificar em producao: `SELECT relname, reloptions FROM pg_class WHERE relname = 'vw_relatorio_leads'`. Em seguida versionar a view numa migration com `WITH (security_invoker = true)` e um `GRANT` explicito, e mover o export para o backend, atras de `authMiddleware` + verificacao de papel + filtro por `aces_id` (o padrao de `/api/collections/imports` em `api-server.ts:1114-1128`). Remover a condicao `acesId === 535` do frontend, que e regra de negocio hardcoded e nao controle de acesso.

Critérios de aceite

- [] `vw_relatorio_leads` definida em `supabase/migrations/` com `security_invoker = true`.
- [] Consulta a `vw_relatorio_leads` com JWT de um tenant retorna apenas linhas daquele `aces_id`.
- [] Export passa por endpoint do backend com verificacao de papel; nenhuma chamada direta do frontend a view.
- [] A constante 535 nao aparece mais como criterio de acesso no frontend.

3. IDOR

3 achado(s)

CRITICA

F01 — Emissao de token de webhook sem nenhuma autenticao (/webhook/login)

Localizacao

Project/IA/api-server.ts:841-853

Project/IA/rb-connection-service.ts:149-160

Project/IA/rb-connection-service.ts:162-183

Trecho do codigo

```
app.post(
  "/webhook/login",
  asyncHandler(async (req, res) => {
    const rbAcesId = Number(req.body.aces_id);
    const empId = Number(req.body.emp_id);
    if (!Number.isInteger(rbAcesId) || rbAcesId <= 0 || !Number.isInteger(empId) || empId <= 0) {
      throw new HttpError(400, "aces_id e emp_id sao obrigatorios e devem ser validos");
    }
    const connection = await rbConnectionService.authenticate({ rbAcesId });
    if (!connection) throw new HttpError(404, "aces_id RB nao cadastrado");
    const { token, exp } = rbConnectionService.signWebhookToken(connection, empId);
    res.json({ token, aces_id: rbAcesId, exp: String(exp) });
  }),
);
```

Por que e explorável

A rota nao passa por authMiddleware nem valida qualquer segredo, assinatura, mTLS ou allowlist de IP. A unica 'credencial' exigida e um inteiro positivo (aces_id) que a propria rota confirma existir respondendo 404 quando nao existe — ou seja, o endpoint funciona como oraculo de enumeracao. rbConnectionService.authenticate() busca a conexao apenas por rb_aces_id e is_active, e signWebhookToken() assina um JWT HS256 valido por 300s contendo connection_id e internal_aces_id do tenant descoberto. Esse token e aceito por POST /webhook/image e POST /webhook/delete (api-server.ts:990 e 1009), que sao verificados por verifyWebhookToken().

Impacto

Tomada de controle das operacoes de visagismo de qualquer tenant, por atacante anonimo: POST /webhook/delete desativa itens do catalogo (visagism_catalog_items) e APAGA objetos do Supabase Storage do tenant (rb-visagism-service.ts:275-285); POST /webhook/image dispara analise de imagem por LLM debitada do orcamento de IA da conta da vitima. Enumeracao de inteiros pequenos revela quais aces_id RB existem.

Condicao de explorabilidade

Nenhuma. Explorável em qualquer deploy que exponha o backend, sem autenticao previa.

Correcao sugerida

Exigir uma credencial real na emissao: segredo compartilhado por conexao (header validado em tempo constante contra collections/source_credentials ou rb.connections), ou assinatura HMAC do corpo, ou mTLS/allowlist de IP do parceiro RB. Enquanto isso, responder sempre 401 generico (sem distinguir aces_id inexistente de credencial invalida) e aplicar rate limit por IP, reaproveitando consume_webhook_rate_limit, que ja existe no schema collections.

Critérios de aceite

- [] POST /webhook/login sem credencial valida responde 401 com corpo generico, para aces_id existente e inexistente.
- [] Nenhum token e emitido sem apresentacao de credencial por conexao.
- [] Rate limit por IP aplicado em /webhook/login (reaproveitar consume_webhook_rate_limit).

[] Teste automatizado cobrindo: (a) sem credencial -> 401; (b) credencial de outra conexao -> 401; (c) credencial correta -> 200 com token.

[] Tokens emitidos antes da correcao invalidados por rotacao de RB_WEBHOOK_JWT_SECRET.

ALTA**F02 — Validacao do webhook da Evolution falha aberta quando o segredo nao esta configurado**

Localizacao

Project/IA/sdr-agent-gemini.ts:15828-15835

Project/IA/api-server.ts:628-637

Project/IA/api-server.ts:832-833

Project/IA/api-server.ts:460

Trecho do codigo

```
// sdr-agent-gemini.ts:15828
validateWebhookSecret(headerValue?: string | null) {
  if (!this.config.evolutionWebhookSecret) {
    return true; // <-- fail-open
  }
  const provided = headerValue?.replace(/^Bearer\s+/i, "").trim();
  return provided === this.config.evolutionWebhookSecret; // comparacao nao constante
}
```

Por que e explorável

EVOLUTION_WEBHOOK_SECRET e opcional (api-server.ts:460 usa process.env sem requireEnv). Quando a variavel esta vazia, validateWebhookSecret() devolve true para qualquer requisicao e o webhookHandler (api-server.ts:628) segue direto para manager.processEvolutionWebhook(). Diferente do handler da Gupshup, que barra a ausencia de segredo em producao ('if (!configuredSecret && process.env.NODE_ENV === "production") throw 503', api-server.ts:710-712), aqui nao existe guarda de ambiente. A comparacao tambem usa === em vez de timingSafeEqual, ao contrario dos webhooks Meta/Instagram.

Impacto

Injecao anonima de mensagens de entrada falsas em qualquer tenant via POST /webhook/evolution ou /api/webhook/evolution: criacao/alteracao de leads, disparo do agente de IA (com custo de LLM), envio de respostas ao WhatsApp e movimentacao de etapas do funil. Envenenamento do historico de conversa e possivel prompt injection indireta no agente.

Condicao de explorabilidade

Requer EVOLUTION_WEBHOOK_SECRET vazio/ausente no ambiente. Nao ha nada no codigo que impeca esse estado.

Correcao sugerida

Fazer a validacao falhar fechada: retornar false quando o segredo nao estiver configurado e adicionar assercao de startup que aborte o processo se EVOLUTION_WEBHOOK_SECRET estiver ausente com NODE_ENV=production (ou promover a requireEnv). Trocar === por crypto.timingSafeEqual sobre buffers de mesmo tamanho, como ja e feito em meta-webhook.ts:56-70.

Critérios de aceite

[] Com EVOLUTION_WEBHOOK_SECRET ausente e NODE_ENV=production, o processo nao sobe (ou o webhook responde 503).

[] validateWebhookSecret retorna false quando o segredo nao esta configurado.

[] Comparacao do segredo feita em tempo constante.

[] Teste cobrindo: sem segredo configurado -> requisicao rejeitada; segredo errado -> 401; segredo correto -> 202.

MEDIA

F06 — Upsert por id fornecido pelo cliente permite sequestro de linhas de outro tenant

Localizacao

Project/IA/api-server.ts:1345-1362

Project/IA/api-server.ts:1364-1395

Project/IA/api-server.ts:1292-1316

Trecho do codigo

```
// api-server.ts:1345 — /api/collections/bindings
const { data, error } = await collectionApi.collectionService.collections
  .from("agent_source_bindings").upsert({
    id: asString(req.body.id) ?? undefined, // <-- id arbitrario do cliente
    aces_id: context.acesId,
    agent_tool_id: String(req.body.agentToolId ?? ""),
    source_connection_id: String(req.body.sourceConnectionId ?? ""),
    ...
  }).select("*").single();

// mesmo padrao em journey_rules (1364) e mapping_profiles (1292)
```

Por que e explorável

collectionApi.collectionService.collections e um client service_role (collections/collection-service.ts:62), logo RLS nao se aplica; o unico limite e o filtro manual. O upsert do supabase-js usa a chave primaria como conflict target por padrao, e a PK dessas tabelas e apenas id. Passando o id de uma linha de outro tenant, o UPDATE acerta aquela linha e reescreve seu aces_id para o do atacante. As FKs compostas (agent_tool_id, aces_id) e (source_connection_id, aces_id) — 20260903205231_collections_core_v1.sql:115-120 — impedem apontar para objetos de outro tenant, mas nao impedem a linha alheia de ser movida.

Impacto

Escrita destrutiva cross-tenant: um ADMIN do tenant A remove silenciosamente um binding, uma journey_rule ou um mapping_profile do tenant B (a linha passa a pertencer a A). Para journey_rules isso interrompe a jornada de cobranca da vitima. Nao ha leitura de dados alheios, apenas perda de configuracao. Requer conhecer/adivinhar um UUID.

Condicao de explorabilidade

Requer papel ADMIN em algum tenant e conhecimento do UUID da linha alvo.

Correcao sugerida

Nao aceitar id do cliente em criacao. Para atualizacao, separar as rotas: PATCH /:id que faca UPDATEeq('id', id).eq('aces_id', context.acesId) e retorne 404 quando nao afetar linha — exatamente o padrao ja usado em /api/collections/sources/:id (api-server.ts:1064-1081). Alternativamente, declarar o onConflict como a chave natural do negocio (ex.: 'agent_tool_id,source_connection_id,creditor_external_id') e nunca a PK.

Critérios de aceite

- [] POST/PATCH das tres rotas nunca aceitam id arbitrario para criar linha nova.
- [] Atualizacao ocorre por UPDATE com .eq('aces_id', context.acesId) e retorna 404 quando nao ha match.
- [] Teste: id de linha de outro tenant -> 404 e a linha original permanece intacta com o aces_id original.
- [] Nenhum upsert no schema collections usa a PK como conflict target com id vindo do corpo.

4. Chaves expostas

3 achado(s)

ALTA

F04 — Senha de root do VPS de producao em texto puro em scripts do projeto

Localizacao

`scripts/get_vps_logs.js:36-39``scripts/check_vps_git.js:39-45``scripts/search_vps_logs.js:36-39`

Trecho do codigo

```
// scripts/get_vps_logs.js:36
}).connect({
  host: "72.60.251.89",
  port: 22,
  username: "root",
  password: "<SENHA REAL REDIGIDA NESTE RELATORIO>"
});

// A senha aparece literalmente na linha 39 do arquivo.
// Foi omitida aqui de proposito: este relatorio circula, a credencial nao deve.
```

Por que e explorável

Credencial de root SSH de host de producao gravada literalmente em tres arquivos da arvore do projeto. Verificado que os tres estao cobertos por `.gitignore:68 (/scripts/*vps*.js)` e que nunca foram commitados (`'git log --all'` vazio para esses caminhos) — mas a protecao e apenas um glob de nome de arquivo. Renomear para `scripts/logs.js`, copiar o trecho para outro script, colar em um issue/PR ou usar `'git add -f'` derrama a senha no historico. Alem disso, o par usuario/senha indica que autenticao por senha para root esta habilitada no SSH do host.

Impacto

Comprometimento total do host: acesso root ao VPS, que executa a Docker Swarm do backend e portanto carrega `SUPABASE_SERVICE_ROLE_KEY`, `GEMINI_API_KEY`, `OPENAI_API_KEY`, `EVOLUTION_API_KEY`, `INSTAGRAM_APP_SECRET`, `INSTAGRAM_TOKEN_ENCRYPTION_KEY`, `COLLECTION_SECRETS_ENCRYPTION_KEY` e `RB_WEBHOOK_JWT_SECRET` no ambiente dos servicos. Com a `service_role` em maos, todo o isolamento multi-tenant cai.

Condicao de explorabilidade

Arquivos ignorados pelo Git e presentes apenas no disco local; o risco e de vazamento por copia/renomeacao e de exposicao da maquina do desenvolvedor.

Correcao sugerida

Rotacionar a senha de root imediatamente. Migrar os tres scripts para autenticao por chave (`privateKey` lida de caminho fora do repositorio) ou, melhor, para o binario `ssh` com um bloco em `~/.ssh/config`, sem host nem credencial no codigo — host e usuario vindos de variaveis de ambiente. Desabilitar `PermitRootLogin/PasswordAuthentication` no `sshd` do host. Ampliar o `.gitignore` de um glob de nome para uma regra por diretorio (ex.: `/scripts/local/`) e adicionar um hook de pre-commit com varredura de segredos.

Critérios de aceite

- ☐ Senha de root do VPS rotacionada e a antiga invalidada.
- ☐ Nenhum dos tres scripts contem host, usuario ou senha literais; credenciais vem de `env`/arquivo externo.
- ☐ `sshd` do host com `PasswordAuthentication` no e `PermitRootLogin prohibit-password` (ou no).
- ☐ Hook de pre-commit com varredura de segredos ativo no repositorio.
- ☐ `grep -rn 'password:' scripts/` nao retorna credencial literal.

MEDIA

F09 — Segredo default publico no docker-compose e ausencia de validacao de startup

Localizacao

`docker-compose.yml:25`

`Project/IA/api-server.ts:274-280`

`.env.local.example:57-58`

Trecho do codigo

```
# docker-compose.yml:25
- RB_WEBHOOK_JWT_SECRET=${RB_WEBHOOK_JWT_SECRET:-local-only-rb-webhook-secret}

// api-server.ts:274 - requireEnv so checa presenca, nao qualidade
function requireEnv(name: string) {
  const value = process.env[name];
  if (!value) {
    throw new Error(`Variavel de ambiente obrigatoria ausente: ${name}`);
  }
  return value;
}
```

Por que e explorável

O valor 'local-only-rb-webhook-secret' esta versionado e vira segredo real de assinatura sempre que RB_WEBHOOK_JWT_SECRET nao for exportado no shell que sobe o compose. Esse segredo assina os JWT HS256 de webhook do RB (rb-connection-service.ts:162-183), portanto quem conhece o default forja tokens offline para qualquer connection_id/aces_id, sem nem precisar do /webhook/login. Nao existe validacao de startup que rejeite valores default ou de baixa entropia: requireEnv apenas verifica se a string e nao vazia. Os placeholders 'local-dev-verify-token' e 'local-dev-app-secret' de .env.local.example:57-58 tem o mesmo problema de classe. Nota positiva: o stack de producao (docker-stack.backend.yml) usa \${VAR} puro, sem default, em todos os 18 segredos.

Impacto

Falsificacao completa dos tokens de webhook do RB em qualquer ambiente que herde o default, combinando com F01 para acesso cross-tenant as operacoes de visagismo. Em compose de dev/homolog com dados reais, o impacto e o mesmo de producao.

Condicao de explorabilidade

Explorável apenas em ambientes que subiram sem exportar a variavel. O stack de producao nao tem defaults.

Correcao sugerida

Remover o default do compose (usar \${RB_WEBHOOK_JWT_SECRET:?RB_WEBHOOK_JWT_SECRET obrigatorio}, que faz o compose falhar cedo) e adicionar validacao de startup no backend: uma funcao requireStrongSecret(name) que rejeite valores de uma denylist de defaults conhecidos e exija comprimento minimo (≥ 32 bytes) quando NODE_ENV=production. Rotacionar o segredo em todos os ambientes que possam ter subido com o default.

Critérios de aceite

- [] Nenhum \${VAR:-valor} com valor de segredo nos arquivos compose/stack (apenas flags e nomes de modelo).
- [] Backend aborta o startup em producao se algum segredo for default conhecido ou tiver menos de 32 bytes.
- [] Teste unitario da validacao de startup cobrindo default conhecido, segredo curto e segredo valido.
- [] RB_WEBHOOK_JWT_SECRET rotacionado em todos os ambientes.

MEDIA

F10 — Segredos de terceiros commitados no historico do Git

Localizacao

referencias/Cobranca API/Curl.txt (commit 02333eb e seguintes)

referencias/eMetizaun/CONFIGURACAO_SUPABASE.md (commit 02333eb e seguintes)

Trecho do codigo

```
# git show 02333eb:referencias/Cobranca API/Curl.txt
# -> Bearer JWT da API RB. Payload decodificado:
# {"iss":"<id do emissor redigido>", "exp":"2026-07-08T14:42:19", ...,
#   "secrets":"eyJzZWNyZXRzIjoi..."}

# git show 02333eb:referencias/eMetizaun/CONFIGURACAO_SUPABASE.md
# -> chave Supabase de OUTRO projeto. Payload decodificado:
# {"iss":"supabase", "ref":"hkqrgomafbohittsdnea", "role":"anon", ...}
```

Por que e explorável

Os arquivos foram removidos do HEAD (nao aparecem em 'git ls-tree -r HEAD'), mas continuam recuperaveis em qualquer clone via 'git show <commit>:<path>'. Verificado que nenhum outro arquivo versionado contem segredo (varredura por AlzaSy*, sk-*, JWT HS256 e blocos PRIVATE KEY em todo o 'git ls-files' voltou vazia) e que todos os .env* estao corretamente ignorados. Os dois valores sao de risco baixo em si — o JWT do RB tem exp em 2026-07-08 (expirado) e a chave Supabase e uma anon key (publica por design) de um projeto de terceiro. O problema real e de processo: segredos chegaram ao historico e nao houve rotacao nem limpeza.

Impacto

Direto: baixo — token RB expirado e anon key publica de outro projeto. Indireto: o campo 'secrets' embutido no JWT do RB pode conter material reutilizavel, e o padrao demonstra ausencia de barreira de commit para segredos. O mesmo caminho poderia ter levado uma service_role ao historico.

Condicao de explorabilidade

Requer acesso de leitura ao repositorio (clone). Ambos os valores tem impacto direto limitado.

Correcao sugerida

Confirmar com o parceiro RB que o token esta revogado (nao apenas expirado). Adotar varredura de segredos no pre-commit e na CI (gitleaks ou equivalente). Avaliar a limpeza do historico com git filter-repo para os dois caminhos, coordenando o rewrite com quem tem clones. Manter a regra de nunca versionar diretorios de referencia com material de integracao.

Critérios de aceite

- [] Token RB confirmado como revogado pelo parceiro.
- [] Varredura de segredos rodando no pre-commit e na CI, com falha bloqueante.
- [] Decisao registrada sobre limpeza do historico (executada ou aceita como risco, com justificativa).
- [] gitleaks detect no repositorio nao retorna achados novos.

5. Inputs sem tratamento (XSS)**4 achado(s)****BAIXA****F11 — XSS refletido no challenge de verificacao dos webhooks Meta e Instagram****Localizacao****Project/IA/api-server.ts:757-767****Project/IA/api-server.ts:784-806****Project/IA/meta-webhook.ts:44-54****Trecho do codigo**

```
// api-server.ts:757
app.get("/api/webhook/meta", (req, res) => {
  const challenge = metaWebhookProcessor.verifyChallenge(req.query as Record<string, unknown>);
  if (!challenge) { res.status(403).send("Forbidden"); return; }
  res.status(200).send(challenge); // Express: string -> text/html
});

// api-server.ts:805 — mesmo padrao com hub.challenge cru
res.status(200).send(challenge);
```

Por que e explorável

Confirmado empiricamente com o Express 5.2.1 instalado no projeto: `res.send(string)` define `Content-Type: text/html; charset=utf-8`. O valor de `hub.challenge` vem direto da query string e e devolvido sem escape, sem `Content-Type` explicito e sem `X-Content-Type-Options`. `GET /api/webhook/instagram?hub.mode=subscribe&hub.verify_token=<token>&hub.challenge=<script>...` executa script na origem do backend. A exploracao depende de conhecer `META_WEBHOOK_VERIFY_TOKEN` ou `INSTAGRAM_WEBHOOK_VERIFY_TOKEN` — valores que `.env.local.example:57` sugere como `'local-dev-verify-token'`, do tipo que costuma sobreviver a homologacao.

Impacto

Execucao de script na origem do backend. Como o frontend guarda a sessao em `localStorage` de outra origem, nao ha roubo direto de token; o impacto pratico e phishing convincente hospedado no dominio da API e abuso de qualquer coisa ligada a essa origem.

Condicao de explorabilidade

Requer conhecer o verify token do webhook (segredo compartilhado com a Meta).

Correcao sugerida

Responder com `Content-Type` explicito e nao-HTML: `res.type('text/plain').send(challenge)`, ou `res.set('Content-Type','text/plain; charset=utf-8')`. Validar que challenge casa com `/^[A-Za-z0-9_-]{1,256}$/` antes de refletir e adicionar `X-Content-Type-Options: nosniff` (o endpoint de midia do Instagram em `api-server.ts:815` ja faz isso e serve de referencia).

Critérios de aceite

- [] Resposta dos dois endpoints GET de verificacao usa `Content-Type text/plain`.
- [] challenge validado por allowlist de caracteres antes de ser refletido.
- [] Header `X-Content-Type-Options: nosniff` presente nas respostas.
- [] Teste de rota confirmando que payload com `< >` nao e devolvido como HTML.

BAIXA**F12 — Open redirect por URL protocolo-relativa no returnPath do OAuth do Instagram****Localizacao**

Project/IA/instagram-service.ts:89-92

Project/IA/instagram-service.ts:151-156

Project/IA/api-server.ts:3822-3852

Trecho do codigo

```
// instagram-service.ts:89
function sanitizeReturnPath(value: string | null | undefined) {
  const normalized = value?.trim() || "/admin?section=instances";
  return /^\/[A-Za-z0-9\_\?&=.%-]*$/.test(normalized) ? normalized : "/admin?section=instances";
}

// "//evil.com" passa no regex (comeca com / e usa apenas caracteres permitidos)
// instagram-service.ts:153
const url = new URL(sanitizeReturnPath(returnPath), `${base}/`);
// new URL("//evil.com", "https://crm.example.com/") === "https://evil.com/"
```

Por que e explorável

Verificado em Node: a string '/evil.com' satisfaz o regex de sanitizacao (inicia com '/' e todos os caracteres seguintes estao na classe permitida) e, resolvida com new URL contra a base, produz uma URL de outra origem, porque '/' e interpretado como URL protocolo-relativa. O valor sai de beginOAuth (req.body.returnPath, api-server.ts:3810), e persistido no state assinado e usado no res.redirect(303, ...) do callback.

Impacto

Redirecionamento para dominio externo a partir de uma URL do backend confiavel. O returnPath vem do proprio usuario autenticado que inicia o fluxo, entao o cenario principal e auto-redirecionamento; o valor real esta em usar o dominio da API como trampolim em phishing e em burlar filtros que confiam no dominio de origem.

Condicao de explorabilidade

Requer que o proprio usuario inicie o fluxo OAuth com o returnPath manipulado.

Correcao sugerida

Rejeitar caminhos que comecem com '/' (ou com '\\'): exigir /^(?!\/)[A-Za-z0-9_\?&=.%-]*\$/ e, como defesa em profundidade, validar depois do new URL que url.origin === new URL(base).origin, caindo no default quando divergir.

Critérios de aceite

- [] sanitizeReturnPath devolve o default para '/evil.com', '//evil.com' e '\\evil.com'.
- [] buildFrontendRedirect confirma que a origem final e a do frontend configurado.
- [] Teste unitario com os vetores acima.

BAIXA

F16 — Sessao Supabase persistida em localStorage

Localizacao

src/integrations/supabase/client.ts:15-24

Trecho do codigo

```
export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
  db: { schema: 'crm' },
  auth: {
    storage: localStorage, // access + refresh token legiveis por JS
    persistSession: true,
    autoRefreshToken: true,
  }
});
```

Por que e explorável

Access token e refresh token ficam em localStorage, acessíveis a qualquer script da origem. A auditoria não encontrou nenhum sink de XSS explorável no frontend (não existe innerHTML, eval nem new Function em src/, e os dois dangerouslySetInnerHTML são CSS estático), portanto isto é amplificação de risco, não vulnerabilidade ativa. Registrado porque o projeto não tem biblioteca de sanitização instalada, o que aumenta a chance de um sink aparecer em código futuro.

Impacto

Se um XSS surgir no frontend, o atacante extrai o refresh token e mantém acesso persistente a conta mesmo depois do fechamento da aba, com o tenant e o papel da vítima.

Condição de explorabilidade

Sem impacto isolado; depende de um XSS futuro no frontend.

Correção sugerida

Ao menos adicionar Content-Security-Policy restritiva no index.html/headers da Vercel para reduzir a superfície, e incluir DOMPurify como dependência obrigatória antes de qualquer futura renderização de HTML. Migrar para armazenamento em cookie httpOnly exige um proxy de autenticação no backend e deve ser avaliado como trabalho maior.

Critérios de aceite

- ☐ CSP restritiva publicada para o frontend (sem unsafe-inline em script-src).
- ☐ Decisão registrada sobre manter localStorage ou migrar para cookie httpOnly.
- ☐ Regra de lint bloqueando novos dangerouslySetInnerHTML sem sanitização.

INFORMATIVA F18 — Sink de CSS via dangerouslySetInnerHTML em componente de chart (latente)

Localização

src/components/ui/chart.tsx:61-88

src/components/ui/material-ui-dropdown-menu.tsx:193-215

Trecho do código

```
// chart.tsx:70 — id e cores entram num <style> sem escape
<style
  dangerouslySetInnerHTML={{
    __html: Object.entries(THEMES).map(([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig.map(([key, itemConfig]) => {
  const color = itemConfig.theme?.[theme] || itemConfig.color;
  return color ? `--color-${key}: ${color};` : null;
}).join("\n")}
  `).join("\n"),
  }}
/>
```

Por que e explorável

São os dois únicos usos de dangerouslySetInnerHTML no projeto. O de material-ui-dropdown-menu.tsx:198 é CSS 100% estático, sem interpolação — sem risco. O de chart.tsx:70 interpola id e cores num bloco <style>, mas hoje id vem de React.useId() (chart.tsx:40-41) e as cores vem de ChartConfig definido em código; confirmado que ChartContainer não é usado em nenhum lugar fora de chart.tsx, ou seja, é boilerplate do shadcn/ui atualmente morto. Registrado como informativo porque viraria injeção de CSS no momento em que alguém alimentar o config com dado de banco (por exemplo, cor de etapa ou de tag escolhida pelo usuário).

Impacto

Nenhum hoje. Se alimentado por dado do usuário, permitiria injeção de CSS (exfiltração por seletor de atributo, defacement); não permite execução de script em navegadores atuais dentro de <style>.

Condicao de explorabilidade

Nao explorável no estado atual do codigo: o componente nao e instanciado em nenhuma pagina.

Correcao sugerida

Se o componente voltar a ser usado, validar as cores por allowlist (por exemplo `/^[0-9a-fA-F]{3,8}$/` ou nomes de token conhecidos) e as chaves por `/^[a-zA-Z0-9_]+$/` antes da interpolacao. Caso siga sem uso, remover `chart.tsx` reduz a superficie.

Critérios de aceite

- [] Componente removido, ou cores/chaves validadas por allowlist antes da interpolacao.
- [] Regra de lint impedindo novos `dangerouslySetInnerHTML` sem sanitizacao explicita.

Extra (defesa em profundidade)

3 achado(s)

BAIXA

F13 — Erros crus do banco devolvidos ao cliente no campo details

Localizacao

- Project/IA/api-server.ts:4315-4321
- Project/IA/sdr-agent-gemini.ts:1122-1131
- Project/IA/api-server.ts:2093 (e outras 18 ocorrencias do padrao)

Trecho do codigo

```
// api-server.ts:4315 – handler global devolve details verbatim
app.use((error: unknown, req: Request, res: Response, _next: NextFunction) => {
  if (error instanceof HttpError) {
    return res.status(error.statusCode).json({
      error: error.message,
      details: error.details ?? null,      // <-- objeto de erro do PostgREST
    });
  }
}

// padrao dos call sites (19 ocorrencias):
// throw new HttpError(500, "Nao foi possivel carregar usuarios", error);
```

Por que e explorável

`HttpError` guarda o terceiro argumento em `details` (`sdr-agent-gemini.ts:1122-1131`) e o handler global serializa esse campo na resposta. Em 19 pontos de `api-server.ts` o objeto passado e o erro cru do `supabase-js/PostgREST`, que carrega `message`, `details`, `hint` e `code` — muitas vezes com nome de tabela, coluna, constraint e fragmento de SQL.

Impacto

Divulgacao de informacao: mapeamento do schema interno (tabelas, colunas, constraints, funcoes) para qualquer usuario autenticado, o que facilita a fase de reconhecimento de outros ataques. Sem impacto direto de integridade ou confidencialidade de dados.

Condicao de explorabilidade

Requer sessao autenticada capaz de provocar erro em alguma rota.

Correcao sugerida

Registrar o erro completo no log do servidor e responder apenas com a mensagem tratada mais um `requestId` de correlacao. Se detalhes forem uteis ao cliente, expor uma lista fechada de campos seguros (por exemplo, apenas `code` mapeado para mensagem amigavel) e nunca o objeto cru — em producao, omitir details por completo.

Cr terios de aceite

- [] Respostas de erro em producao nao contem message/details/hint do PostgREST.
- [] Erro completo permanece no log do servidor, correlacionado por requestId devolvido ao cliente.
- [] Nenhum call site passa objeto de erro cru para o construtor de HttpError sem sanitizacao.

BAIXA

F14 — Limite default de corpo JSON em 150 MB**Localizacao****Project/IA/api-server.ts:568-580****Trecho do codigo**

```
app.use(
  express.json({
    limit:
      process.env.JSON_BODY_LIMIT ?? process.env.WEBHOOK_JSON_LIMIT ?? "150mb",
    verify: (req, _res, buf) => { ... },
  }),
);
```

Por que e explorável

O parser global aceita ate 150 MB por requisicao quando nenhuma das duas variaveis esta definida. Esse limite vale para todas as rotas montadas depois, inclusive os webhooks nao autenticados (/webhook/evolution, /api/webhook/gupshup). Cada corpo e materializado em memoria antes de qualquer verificacao de credencial. Rotas especificas fazem melhor: o webhook de collections usa `express.raw({ limit: '1mb' })` (api-server.ts:553-557) e /webhook/image tem guarda de 15 MB (api-server.ts:559-567).

Impacto

Exaustao de memoria/CPU com poucas requisicoes concorrentes, levando a indisponibilidade do backend. Nao ha impacto de confidencialidade.

Condicao de explorabilidade

Nenhuma; alcancavel nas rotas de webhook sem autenticacao.

Correcao sugerida

Reduzir o default global para algo na ordem de 1-2 MB e elevar o limite apenas nas rotas que comprovadamente precisam de payload grande, montando um `express.json` dedicado por rota, como ja e feito em /webhook/image.

Critérios de aceite

- [] Default global do `express.json` em `<= 2mb`.
- [] Rotas que precisam de payload maior tem parser dedicado com limite documentado.
- [] Teste confirmando 413 para corpo acima do limite global em rota comum.

BAIXA

F15 — Comparacao de segredos de webhook fora de tempo constante**Localizacao****Project/IA/sdr-agent-gemini.ts:15834****Project/IA/api-server.ts:705-709****Trecho do codigo**

```
// sdr-agent-gemini.ts:15834 (Evolution)
return provided === this.config.evolutionWebhookSecret;
```

```
// api-server.ts:706 (Gupshup) – também aceita o segredo pela query string
const providedSecret =
  req.header("x-gupshup-secret") || req.header("x-webhook-secret") ||
  asString(req.query.secret);
if (configuredSecret && providedSecret !== configuredSecret) { throw new HttpError(401, ...); }
```

Por que e explorável

Os dois handlers comparam segredos com `===`, que faz curto-circuito no primeiro byte diferente. Os webhooks Meta, Instagram e collections fazem o correto, com `crypto.timingSafeEqual` (meta-webhook.ts:67-70, instagram-webhook.ts:119, collections/webhook-security.ts:51). Agravante no Gupshup: o segredo pode vir em `req.query.secret`, e query strings vazam para access logs, referrers e historico de proxy.

Impacto

Canal lateral de tempo teoricamente explorável para recuperar o segredo byte a byte; na pratica, ruidoso sobre rede. O vazamento do segredo por query string em logs e o risco mais concreto.

Condicao de explorabilidade

Nenhuma para o vazamento em log; o ataque de tempo exige muitas amostras.

Correcao sugerida

Comparar com `crypto.timingSafeEqual` sobre buffers, checando o tamanho antes. Remover `req.query.secret` como fonte aceita, mantendo apenas headers.

Critérios de aceite

- [] Comparacao de segredo de webhook usa `timingSafeEqual` em todos os handlers.
- [] Segredo da Gupshup aceito somente por header.
- [] Teste unitario cobrindo segredo correto, incorreto e de tamanho diferente.

Recomendacoes priorizadas

Prioridade definida por impacto x facilidade de exploracao x esforco de correcao. P1 concentra o que e explorável sem autenticacao previa ou o que expoe credencial de producao.

P1	Imediato (mesma semana)	4 acao(oes)
P1.1	Exigir credencial em POST /webhook/login e rotacionar RB_WEBHOOK_JWT_SECRET F01, F09 — hoje qualquer anonimo emite token valido para qualquer tenant e o segredo pode ser o default publicado.	
P1.2	Rotacionar a senha de root do VPS e migrar os scripts para autenticacao por chave F04 — credencial de producao em texto puro; com root vem a service_role e todo o isolamento cai.	
P1.3	Fazer validateWebhookSecret falhar fechada e abortar o startup sem EVOLUTION_WEBHOOK_SECRET em producao F02 — injecao anonima de mensagens em qualquer tenant se a variavel estiver ausente.	
P1.4	Adicionar autorizacao e vinculo convite<->e-mail na Edge Function send-user-invitation F03 — escalonamento de VENDEDOR para ADMIN reaproveitando um invitationId da propria conta.	
P2	Curto prazo (2 a 4 semanas)	6 acao(oes)
P2.1	Verificar security_invoker de vw_relatorio_leads em producao, versionar a view e mover o export para o backend F08 — no cenario default de Postgres a view ignora a RLS de crm.leads e vazaria a base de todos os tenants.	
P2.2	Adicionar verificacao de papel e posse do agente em /api/agents/:id/tools/rb_billing/run-now F05 — unica rota de tools que nao passa pelo manager; hoje qualquer usuario do tenant dispara cobranca.	
P2.3	Escopar por aces_id a sincronizacao de templates Meta F07 — escrita cross-tenant em meta.whatsapp_templates e uso do token da vitima na Graph API.	
P2.4	Eliminar o upsert por id vindo do cliente nas tres rotas de collections F06 — permite mover linhas de configuracao de outro tenant para a conta do atacante.	
P2.5	Introduzir validacao de startup de qualidade de segredo e remover defaults dos arquivos compose F09 — requireEnv so checa presenca; defaults publicos entram silenciosamente em ambientes reais.	
P2.6	Ativar varredura de segredos no pre-commit e na CI, e tratar o material do historico F04, F10 — nao existe barreira automatica; segredos ja chegaram ao historico uma vez.	
P3	Medio prazo (1 a 2 meses)	7 acao(oes)
P3.1	Parar de devolver o objeto de erro do PostgREST e passar a correlacionar por requestId F13 — 19 call sites expoe tabela, coluna, constraint e fragmento de SQL a qualquer autenticado.	
P3.2	Refletir hub.challenge como text/plain, com allowlist de caracteres e nosniff F11 — XSS refletido na origem do backend para quem conhece o verify token.	
P3.3	Corrigir sanitizeReturnPath para rejeitar '/' e validar a origem final F12 — open redirect por URL protocolo-relativa.	
P3.4	Reduzir o limite global de corpo JSON para <= 2 MB e elevar apenas por rota F14 — 150 MB no parser global, alcancavel em webhook nao autenticado.	
P3.5	Padronizar timingSafeEqual em toda comparacao de segredo e remover o segredo da query string F15 — canal lateral de tempo e vazamento de segredo em access log.	

P3.6	Publicar CSP restritiva para o frontend e adotar DOMPurify como dependencia obrigatoria F16, F18 — sessao em localStorage sem lib de sanitizacao no projeto.
P3.7	Resolver papel na tabela (nao na claim) nas policies mais sensiveis, ou invalidar sessao ao trocar papel F17 — janela de privilegio residual de ate uma expiracao de token apos rebaixamento.

P4	Melhoria continua	4 acao(oes)
P4.1	Criar um wrapper tipado obrigatorio para consultas com service_role, exigindo acesId explicito Transforma em erro de compilacao a classe de falha de F06 e F07, hoje detectavel so em revisao.	
P4.2	Estender as FKs compostas (id, aces_id) do schema collections aos schemas crm, agents e meta O padrao de 20260903205231 e o mais forte do repositorio e deveria ser a regra, nao a excecao.	
P4.3	Adicionar teste de contrato que percorra as rotas e falhe quando faltar authMiddleware ou guarda de papel Fecharia por construcao o tipo de lacuna de F05.	
P4.4	Remover o componente chart.tsx, hoje sem uso, ou validar suas cores por allowlist F18 — reduz superficie de injecao de CSS.	

Anexo — cobertura da revisao

Registro do que foi efetivamente percorrido, para tornar a cobertura auditavel.

Superficie	Volume revisado	Metodo
Rotas Express (Project/IA/api-server.ts)	167 registros de rota, 100%	Leitura sequencial do arquivo inteiro (4.330 linhas), nao amostragem
Metodos de servico com AuthContext (sdr-agent-gemini.ts)	53 metodos, 100%	Extracao programatica + classificacao por guarda de posse
Rotas sem authMiddleware	13 endpoints	Enumeracao e verificacao individual do controle compensatorio
Migrations SQL (supabase/migrations)	173 arquivos	Levantamento de ENABLE RLS, CREATE POLICY, GRANT/REVOKE, security_invoker e FKs compostas
Tabelas/views consultadas direto pelo frontend	23 objetos	Cruzamento com a lista de RLS habilitada e com as policias por operacao
Edge Functions (Deno)	3 funcoes + config.toml	Leitura integral e checagem de verify_jwt vs. verificacao interna
Arquivos de deploy	docker-compose.yml, docker-stack.backend.yml, Dockerfile	Inspecao de defaults \${VAR:-...} e de injecao de segredo
Historico do Git	Todos os commits alcancaveis	git grep por JWT/AlzaSy/sk-/PRIVATE KEY + decodificacao dos payloads encontrados
Bundle compilado	dist/assets/*.js	Varredura por service_role, JWT e chaves de API
Sinks de XSS no frontend	Todo o src/	Busca por dangerouslySetInnerHTML, innerHTML, eval, new Function, href/src dinamicos

Categorias nao aplicaveis: nenhuma das cinco categorias solicitadas ficou fora de escopo. O projeto tem frontend (React), backend proprio (Express), banco com RLS (Supabase/Postgres), arquivos de deploy (Docker/Swarm/Vercel) e funcoes serverless, portanto as cinco foram avaliadas. Nao existe ORM classico (Prisma/TypeORM/Sequelize) nem SQL concatenado no codigo da aplicacao: todo o acesso passa por PostgREST via supabase-js ou por RPCs parametrizadas, de modo que **SQL injection classica nao se aplica** a esta stack — verificado por ausencia de construcao dinamica de query no codigo TypeScript.

Issues para o GitHub

Texto completo de cada issue em Markdown, pronto para copiar e colar. Cada issue esta delimitada entre --- ISSUE n --- e --- FIM ISSUE n ---. Achados triviais do mesmo tema foram agrupados numa issue unica para nao gerar ruido no backlog: F11+F12 (reflexao/redirecionamento na borda HTTP), F13+F14+F15 (endurecimento da borda do backend) e F16+F18 (endurecimento de XSS no frontend).

Total: 14 issues cobrindo os 18 achados.

ISSUE 1 — [Seguranca] Emissao de token de webhook sem nenhuma autenticacao (/webhook/login)

--- ISSUE 1 ---

Titulo

[Seguranca] Emissao de token de webhook sem nenhuma autenticacao (/webhook/login)

Labels

`security`, `severity:critical`

Severidade

CRITICA (achado F01 da auditoria de 04 de setembro de 2026)

Problema

A rota nao passa por authMiddleware nem valida qualquer segredo, assinatura, mTLS ou allowlist de IP. A unica 'credencial' exigida e um inteiro positivo (aces_id) que a propria rota confirma existir respondendo 404 quando nao existe – ou seja, o endpoint funciona como oraculo de enumeracao. rbConnectionService.authenticate() busca a conexao apenas por rb_aces_id e is_active, e signWebhookToken() assina um JWT HS256 valido por 300s contendo connection_id e internal_aces_id do tenant descoberto. Esse token e aceito por POST /webhook/image e POST /webhook/delete (api-server.ts:990 e 1009), que sao verificados por verifyWebhookToken().

Evidencia

- `Project/IA/api-server.ts:841-853`
- `Project/IA/rb-connection-service.ts:149-160`
- `Project/IA/rb-connection-service.ts:162-183`

```
``ts
app.post(
  "/webhook/login",
  asyncHandler(async (req, res) => {
    const rbAcesId = Number(req.body.aces_id);
    const empId = Number(req.body.emp_id);
    if (!Number.isInteger(rbAcesId) || rbAcesId <= 0 || !Number.isInteger(empId) || empId <= 0) {
      throw new HttpError(400, "aces_id e emp_id sao obrigatorios e devem ser validos");
    }
    const connection = await rbConnectionService.authenticate({ rbAcesId });
    if (!connection) throw new HttpError(404, "aces_id RB nao cadastrado");
    const { token, exp } = rbConnectionService.signWebhookToken(connection, empId);
    res.json({ token, aces_id: rbAcesId, exp: String(exp) });
  }),
);
``
```

Impacto

Tomada de controle das operacoes de visagismo de qualquer tenant, por atacante anonimo: POST /webhook/delete desativa itens do catalogo (visagism_catalog_items) e APAGA objetos do Supabase Storage do tenant (rb-visagism-service.ts:275-285); POST /webhook/image dispara analise de imagem por LLM debitada do orcamento de IA da conta da vitima. Enumeracao de inteiros pequenos revela quais aces_id RB existem.

Condicao de explorabilidade

Nenhuma. Explorável em qualquer deploy que exponha o backend, sem autenticacao previa.

Correcao sugerida

Exigir uma credencial real na emissao: segredo compartilhado por conexao (header validado em tempo constante contra collections/source_credentials ou rb.connections), ou assinatura HMAC do corpo, ou mTLS/allowlist de IP do parceiro RB. Enquanto isso, responder sempre 401 generico (sem distinguir aces_id inexistente de credencial invalida) e aplicar rate limit por IP, reaproveitando consume_webhook_rate_limit, que ja existe no schema collections.

Criterios de aceite

- [] POST /webhook/login sem credencial valida responde 401 com corpo generico, para aces_id existente e inexistente.
- [] Nenhum token e emitido sem apresentacao de credencial por conexao.
- [] Rate limit por IP aplicado em /webhook/login (reaproveitar consume_webhook_rate_limit).
- [] Teste automatizado cobrindo: (a) sem credencial -> 401; (b) credencial de outra conexao -> 401; (c) credencial correta -> 200 com token.
- [] Tokens emitidos antes da correcao invalidados por rotacao de RB_WEBHOOK_JWT_SECRET.

--- FIM ISSUE 1 ---

ISSUE 2 — [Seguranca] Validacao do webhook da Evolution falha aberta quando o segredo nao esta configurado

--- ISSUE 2 ---

Titulo

[Seguranca] Validacao do webhook da Evolution falha aberta quando o segredo nao esta configurado

Labels

`security`, `severity:high`

Severidade

ALTA (achado F02 da auditoria de 04 de setembro de 2026)

Problema

EVOLUTION_WEBHOOK_SECRET e opcional (api-server.ts:460 usa process.env sem requireEnv). Quando a variavel esta vazia, validateWebhookSecret() devolve true para qualquer requisicao e o webhookHandler (api-server.ts:628) segue direto para manager.processEvolutionWebhook(). Diferente do handler da Gupshup, que barra a ausencia de segredo em producao ('if (!configuredSecret && process.env.NODE_ENV === "production") throw 503', api-server.ts:710-712), aqui nao existe guarda de ambiente. A comparacao tambem usa === em vez de timingSafeEqual, ao contrario dos webhooks Meta/Instagram.

Evidencia

- `Project/IA/sdr-agent-gemini.ts:15828-15835`
- `Project/IA/api-server.ts:628-637`
- `Project/IA/api-server.ts:832-833`
- `Project/IA/api-server.ts:460`

```ts

```
// sdr-agent-gemini.ts:15828
validateWebhookSecret(headerValue?: string | null) {
 if (!this.config.evolutionWebhookSecret) {
 return true; // <-- fail-open
 }
 const provided = headerValue?.replace(/^Bearer\s+/i, "").trim();
 return provided === this.config.evolutionWebhookSecret; // comparacao nao constante
}
```

**## Impacto**

Injecao anonima de mensagens de entrada falsas em qualquer tenant via POST /webhook/evolution ou /api/webhook/evolution: criacao/alteracao de leads, disparo do agente de IA (com custo de LLM), envio de respostas ao WhatsApp e movimentacao de etapas do funil. Envenenamento do historico de conversa e possivel prompt injection indireta no agente.

```

Condicao de explorabilidade

Requer EVOLUTION_WEBHOOK_SECRET vazio/ausente no ambiente. Nao ha nada no codigo que impeca esse estado.

Correcao sugerida

Fazer a validacao falhar fechada: retornar false quando o segredo nao estiver configurado e adicionar assercao
de startup que aborte o processo se EVOLUTION_WEBHOOK_SECRET estiver ausente com NODE_ENV=production (ou
promover a requireEnv). Trocar === por crypto.timingSafeEqual sobre buffers de mesmo tamanho, como ja e feito
em meta-webhook.ts:56-70.

Criterios de aceite

- [] Com EVOLUTION_WEBHOOK_SECRET ausente e NODE_ENV=production, o processo nao sobe (ou o webhook responde
503).
- [] validateWebhookSecret retorna false quando o segredo nao esta configurado.
- [] Comparacao do segredo feita em tempo constante.
- [] Teste cobrindo: sem segredo configurado -> requisicao rejeitada; segredo errado -> 401; segredo correto ->
202.

--- FIM ISSUE 2 ---

```

## ISSUE 3 — [Seguranca] Edge Function send-user-invitation sem autorizacao e sem vinculo convite<->e-mail

```

--- ISSUE 3 ---

Titulo

[Seguranca] Edge Function send-user-invitation sem autorizacao e sem vinculo convite<->e-mail

Labels

`security`, `severity:high`

Severidade

ALTA (achado F03 da auditoria de 04 de setembro de 2026)

Problema

A funcao nao chama auth.getUser(), nao verifica papel e nao compara o parametro email com invitation.email. O
convite e buscado com service_role apenas por id, sem filtro por tenant. verify_jwt = true (config.toml:51)
garante somente que exista um JWT valido do projeto – qualquer usuario autenticado serve, de qualquer papel e
de qualquer tenant. A policy inv_select (20260411154500_fix_crm_rls_recursion.sql:105-107) permite a qualquer
usuario autenticado ler os convites do seu proprio aces_id, incluindo os de role ADMIN, o que fornece um
invitationId valido.

Evidencia

- `supabase/functions/send-user-invitation/index.ts:38-41`
- `supabase/functions/send-user-invitation/index.ts:43-55`
- `supabase/functions/send-user-invitation/index.ts:74-77`
- `supabase/config.toml:49-51`

```ts
const { email, invitationId } = body ?? {};
if (!email || !invitationId) { /* ... 400 ... */ }

const supabaseAdmin = createClient(
  Deno.env.get("SUPABASE_URL") ?? "",
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? "", // service_role: ignora RLS
);

const { data: invitation } = await supabaseAdmin
  .schema("crm").from("user_invitations")
  .select("*").eq("id", invitationId).single(); // sem filtro por aces_id

await supabaseAdmin.auth.admin.inviteUserByEmail(email, { // email vem do atacante
  data: { name: invitation.name, role: invitation.role,
    aces_id: invitation.aces_id, invitation_id: invitationId },
});

```

```

...

## Impacto

Escalonamento de privilegio: um VENDEDOR ou NENHUM le um invitationId com role ADMIN da sua conta e chama a
funcao com um e-mail que controla; o Supabase envia convite carregando role=ADMIN e aces_id da conta no
app_metadata do novo usuario, resultando em conta ADMIN sob controle do atacante. O parametro email
desacoplado do convite tambem permite redirecionar qualquer convite pendente para um endereco arbitrario.

## Condicao de explorabilidade

Requer apenas uma sessao autenticada qualquer no projeto Supabase e um invitationId valido.

## Correcao sugerida

Na propria funcao: (1) resolver o chamador com um client anon usando o header Authorization e auth.getUser();
(2) carregar o crm.users do chamador e exigir role ADMIN; (3) filtrar o convite por aces_id do chamador; (4)
ignorar o email do corpo e usar exclusivamente invitation.email. Os arquivos buscar-leads/index.ts:56-91 e
import-leads-csv/index.ts:242-276 ja implementam exatamente esse padrao e servem de referencia.

## Criterios de aceite

- [ ] A funcao resolve o chamador via auth.getUser() e rejeita com 401 quando nao ha usuario valido.
- [ ] A funcao rejeita com 403 quando o chamador nao tem role ADMIN.
- [ ] O convite e carregado com filtro .eq('aces_id', callerAcesId); id de outro tenant retorna 404.
- [ ] O destinatario do convite vem de invitation.email; o campo email do corpo e ignorado ou validado como
igual.
- [ ] Teste cobrindo: sem JWT -> 401; JWT de VENDEDOR -> 403; invitationId de outro tenant -> 404; email
divergente -> rejeitado.

--- FIM ISSUE 3 ---

```

ISSUE 4 — [Seguranca] Senha de root do VPS de producao em texto puro em scripts do projeto

```

--- ISSUE 4 ---

## Titulo

[Seguranca] Senha de root do VPS de producao em texto puro em scripts do projeto

## Labels

`security`, `severity:high`

## Severidade

ALTA (achado F04 da auditoria de 04 de setembro de 2026)

## Problema

Credencial de root SSH de host de producao gravada literalmente em tres arquivos da arvore do projeto.
Verificado que os tres estao cobertos por .gitignore:68 (/scripts/*vps*.js) e que nunca foram commitados ('git
log --all' vazio para esses caminhos) – mas a protecao e apenas um glob de nome de arquivo. Renomear para
scripts/logs.js, copiar o trecho para outro script, colar em um issue/PR ou usar 'git add -f' derrama a senha
no historico. Alem disso, o par usuario/senha indica que autenticacao por senha para root esta habilitada no
SSH do host.

## Evidencia

- `scripts/get_vps_logs.js:36-39`
- `scripts/check_vps_git.js:39-45`
- `scripts/search_vps_logs.js:36-39`

```ts
// scripts/get_vps_logs.js:36
}).connect({
 host: "72.60.251.89",
 port: 22,
 username: "root",
 password: "<SENHA REAL REDIGIDA NESTE RELATORIO>"
});

```

```
// A senha aparece literalmente na linha 39 do arquivo.
// Foi omitida aqui de proposito: este relatorio circula, a credencial nao deve.
...

Impacto

Comprometimento total do host: acesso root ao VPS, que executa a Docker Swarm do backend e portanto carrega
SUPABASE_SERVICE_ROLE_KEY, GEMINI_API_KEY, OPENAI_API_KEY, EVOLUTION_API_KEY, INSTAGRAM_APP_SECRET,
INSTAGRAM_TOKEN_ENCRYPTION_KEY, COLLECTION_SECRETS_ENCRYPTION_KEY e RB_WEBHOOK_JWT_SECRET no ambiente dos
servicos. Com a service_role em maos, todo o isolamento multi-tenant cai.

Condicao de explorabilidade

Arquivos ignorados pelo Git e presentes apenas no disco local; o risco e de vazamento por copia/renomeacao e de
exposicao da maquina do desenvolvedor.

Correcao sugerida

Rotacionar a senha de root imediatamente. Migrar os tres scripts para autenticao por chave (privateKey lida de
caminho fora do repositorio) ou, melhor, para o binario ssh com um bloco em ~/.ssh/config, sem host nem
credencial no codigo – host e usuario vindos de variaveis de ambiente. Desabilitar
PermitRootLogin/PasswordAuthentication no sshd do host. Ampliar o .gitignore de um glob de nome para uma regra
por diretorio (ex.: /scripts/local/) e adicionar um hook de pre-commit com varredura de segredos.

Criterios de aceite

- [] Senha de root do VPS rotacionada e a antiga invalidada.
- [] Nenhum dos tres scripts contem host, usuario ou senha literais; credenciais vem de env/arquivo externo.
- [] sshd do host com PasswordAuthentication no e PermitRootLogin prohibit-password (ou no).
- [] Hook de pre-commit com varredura de segredos ativo no repositorio.
- [] grep -rn 'password:' scripts/ nao retorna credencial literal.

--- FIM ISSUE 4 ---
```

## ISSUE 5 — [Seguranca] POST /api/agents/:id/tools/rb\_billing/run-now sem verificacao de papel no servidor

```
--- ISSUE 5 ---

Titulo

[Seguranca] POST /api/agents/:id/tools/rb_billing/run-now sem verificacao de papel no servidor

Labels

`security`, `severity:medium`

Severidade

MEDIA (achado F05 da auditoria de 04 de setembro de 2026)

Problema

Essa e a unica rota de /api/agents/:id/tools/* que nao delega ao manager (que sempre chama ensureAdmin +
getAgentForAccount). Ela chama rbBillingWorker.runNowForAgent diretamente, e o getBindingForAgent
(rb-billing-worker.ts:479-486) filtra apenas por aces_id e agent_id. O isolamento de tenant esta correto; o
que falta e o controle de papel. No frontend, toda a pagina de Agentes esta atras de isAdmin
(src/components/layout/Sidebar.tsx:202), logo a restricao existe so no navegador.

Evidencia

- `Project/IA/api-server.ts:2928-2937`
- `Project/IA/rb-billing-worker.ts:459-462`
- `Project/IA/sdr-agent-gemini.ts:5145-5147`
- `src/services/agentToolsService.ts:380`

...ts
// api-server.ts:2928 – nao chama manager.*, portanto nao passa por ensureAdmin
app.post(
 "/api/agents/:id/tools/rb_billing/run-now",
 authMiddleware,
```

```

 asyncHandler(async (req: AuthenticatedRequest, res) => {
 const agentId = getSingleParam(req.params.id);
 const result = await rbBillingWorker.runNowForAgent(
 req.authContext!.acesId, agentId); // so escopo de tenant, sem papel
 res.status(202).json({ success: true, result });
 }),
);

// contraste – a rota irma valida papel:
// sdr-agent-gemini.ts:5145 async bootstrapRbBilling(...) { this.ensureAdmin(context); ... }
...

Impacto

Qualquer usuario autenticado do tenant, inclusive com role NENHUM ou VENDEDOR, dispara uma rodada de cobranca
fora de hora: envio de mensagens de cobranca a leads reais via WhatsApp, consumo de orcamento de IA e poluicao
dos registros de execucao. Nao ha exposicao de dados de outro tenant.

Condicao de explorabilidade

Requer sessao autenticada no tenant e um agente com a tool rb_billing configurada.

Correcao sugerida

Roteá-la pelo manager, como as demais: adicionar uma verificacao explicita de papel antes da chamada (const
context = req.authContext!; if (context.role !== 'ADMIN') throw new HttpError(403, ...)) e validar a posse do
agente com manager.getAgentForAccount(agentId, context.acesId, context.crmUserId, context.role) – de
preferencia expondo um metodo runRbBillingNow(context, agentId) no AgentManager para manter o padrao.

Criterios de aceite

- [] Chamada com token de usuario VENDEDOR/NENHUM retorna 403.
- [] Chamada com agentId de outro tenant retorna 404.
- [] A rota valida posse do agente pela mesma funcao usada pelas rotas irmas.
- [] Teste de rota cobrindo os tres casos (NENHUM -> 403, VENDEDOR -> 403, ADMIN -> 202).

--- FIM ISSUE 5 ---

```

## ISSUE 6 — [Seguranca] Upsert por id fornecido pelo cliente permite sequestro de linhas de outro tenant

```

--- ISSUE 6 ---

Titulo

[Seguranca] Upsert por id fornecido pelo cliente permite sequestro de linhas de outro tenant

Labels

`security`, `severity:medium`

Severidade

MEDIA (achado F06 da auditoria de 04 de setembro de 2026)

Problema

collectionApi.collectionService.collections e um client service_role (collections/collection-service.ts:62),
logo RLS nao se aplica; o unico limite e o filtro manual. O upsert do supabase-js usa a chave primaria como
conflict target por padrao, e a PK dessas tabelas e apenas id. Passando o id de uma linha de outro tenant, o
UPDATE acerta aquela linha e reescreve seu aces_id para o do atacante. As FKs compostas (agent_tool_id,
aces_id) e (source_connection_id, aces_id) – 20260903205231_collections_core_v1.sql:115-120 – impedem apontar
para objetos de outro tenant, mas nao impedem a linha alheia de ser movida.

Evidencia

- `Project/IA/api-server.ts:1345-1362`
- `Project/IA/api-server.ts:1364-1395`
- `Project/IA/api-server.ts:1292-1316`

```ts
// api-server.ts:1345 – /api/collections/bindings

```

```
const { data, error } = await collectionApi.collectionService.collections
  .from("agent_source_bindings").upsert({
    id: asString(req.body.id) ?? undefined,      // <-- id arbitrario do cliente
    aces_id: context.acesId,
    agent_tool_id: String(req.body.agentToolId ?? ""),
    source_connection_id: String(req.body.sourceConnectionId ?? ""),
    ...
  }).select("").single();

// mesmo padrao em journey_rules (1364) e mapping_profiles (1292)
...

## Impacto

Escrita destrutiva cross-tenant: um ADMIN do tenant A remove silenciosamente um binding, uma journey_rule ou um
mapping_profile do tenant B (a linha passa a pertencer a A). Para journey_rules isso interrompe a jornada de
cobranca da vitima. Nao ha leitura de dados alheios, apenas perda de configuracao. Requer conhecer/adivinhar
um UUID.

## Condicao de explorabilidade

Requer papel ADMIN em algum tenant e conhecimento do UUID da linha alvo.

## Correcao sugerida

Nao aceitar id do cliente em criacao. Para atualizacao, separar as rotas: PATCH /:id que faca UPDATE ...
.eq('id', id).eq('aces_id', context.acesId) e retorne 404 quando nao afetar linha — exatamente o padrao ja
usado em /api/collections/sources/:id (api-server.ts:1064-1081). Alternativamente, declarar o onConflict como
a chave natural do negocio (ex.: 'agent_tool_id,source_connection_id,creditor_external_id') e nunca a PK.

## Criterios de aceite

- [ ] POST/PATCH das tres rotas nunca aceitam id arbitrario para criar linha nova.
- [ ] Atualizacao ocorre por UPDATE com .eq('aces_id', context.acesId) e retorna 404 quando nao ha match.
- [ ] Teste: id de linha de outro tenant -> 404 e a linha original permanece intacta com o aces_id original.
- [ ] Nenhum upsert no schema collections usa a PK como conflict target com id vindo do corpo.

--- FIM ISSUE 6 ---
```

ISSUE 7 — [Seguranca] Sincronizacao de templates Meta nao filtra por tenant

```
--- ISSUE 7 ---

## Titulo

[Seguranca] Sincronizacao de templates Meta nao filtra por tenant

## Labels

`security`, `severity:medium`

## Severidade

MEDIA (achado F07 da auditoria de 04 de setembro de 2026)

## Problema

O handler confere apenas que o chamador e ADMIN e repassa instanceName cru. requireChannel busca o canal so por
instance_name usando client service_role, sem escopo de conta. Todas as rotas irmas fazem o certo:
metaAdminService.upsertChannel passa por requireInstance(acesId, instanceName) (meta-admin-service.ts:182-189)
e listTemplates recebe acesId. Nomes de instancia sao curtos e previsiveis (o script verify-evolution-env.mjs
cita 'prospect', 'Juan', 'Lucas'), o que torna a adivinhacao viavel.

## Evidencia

- `Project/IA/api-server.ts:1488-1508`
- `Project/IA/meta-template-service.ts:106-118`
- `Project/IA/meta-template-service.ts:56-97`

```ts
// api-server.ts:1488 — instanceName vem do corpo e nao e validado contra o tenant
const result = await metaTemplateService.syncTemplatesForInstance(instanceName);
```

```
// meta-template-service.ts:106 — requireChannel sem acesId
private async requireChannel(instanceName: string) {
 const { data, error } = await this.metaClient
 .from("whatsapp_channels")
 .select("id, waba_id, access_token_secret_ref")
 .eq("instance_name", instanceName) // <-- falta .eq("aces_id", acesId)
 .maybeSingle();
 ...

Impacto

Escrita cross-tenant: um ADMIN do tenant A sobrescreve meta.whatsapp_templates e last_template_sync_at do canal do tenant B e provoca chamadas a Graph API da Meta usando o access token do tenant B (consumo de quota e ruído nos logs do parceiro). A resposta devolve apenas { instanceName, mode, synced }, portanto não há leitura direta do conteúdo alheio.

Condição de explorabilidade

Requer papel ADMIN em algum tenant e conhecer o nome de instância da vítima.

Correção sugerida

Propagar o tenant: mudar a assinatura para syncTemplatesForInstance(acesId, instanceName) e adicionar .eq('aces_id', acesId) em requireChannel; ou validar antes no handler com metaAdminService.requireInstance(context.acesId, instanceName). Vale varrer meta-template-service.ts em busca de outras consultas sem escopo de conta.

Critérios de aceite

- [] syncTemplatesForInstance recebe e aplica acesId.
- [] requireChannel filtra por aces_id e instance_name.
- [] Chamada com instanceName de outro tenant retorna 404 e não altera nenhuma linha.
- [] Teste com dois tenants confirmando isolamento.

--- FIM ISSUE 7 ---
```

## ISSUE 8 — [Segurança] Export de relatório de leads restrito apenas no navegador, sobre view sem RLS rastreada

```
--- ISSUE 8 ---

Título

[Segurança] Export de relatório de leads restrito apenas no navegador, sobre view sem RLS rastreada

Labels

`security`, `severity:medium`

Severidade

MEDIA (achado F08 da auditoria de 04 de setembro de 2026)

Problema

A decisão de acesso ao relatório está num if do bundle React: qualquer usuário autenticado pode chamar supabase.from('vw_relatorio_leads').select('*') pelo console do navegador com a chave anon, sem passar por esse if. O que impede o vazamento e exclusivamente a postura de RLS da view — e vw_relatorio_leads NÃO existe em nenhuma das 173 migrations (foi criada fora do versionamento). Views no Postgres nascem com security_invoker = false, isto é, consultam as tabelas-base como o owner e ignoram a RLS de crm.leads. Todas as outras views do projeto tratam isso explicitamente (crm.v_lead_details recebe ALTER VIEW ... SET (security_invoker = true) após cada redefinição — 20260618205533:55, 20260708211239:133, 20260724202355:593, 20260827145250:151).

Evidência

- `src/pages/Leads.tsx:112-137`
- `src/integrations/supabase/client.ts:15-24`

```ts
// src/pages/Leads.tsx:112
const { profile } = await getCrmBackend<CrmProfileResponse>("/api/crm/profile");
```

```
const acesId = profile?.aces_id;

// 3. Verificar se e o aces_id especial (535)
if (acesId === 535) { // <-- gate 100% client-side
  let query = supabase
    .from('vw_relatorio_leads') // view nao definida em supabase/migrations/
    .select('*');
  ...
  const { data: viewData } = await query;
  exportGenericToCSV(viewData, ...);
}

## Impacto

Se a view em producao estiver com security_invoker = false e SELECT concedido a authenticated, qualquer usuario
logado de qualquer tenant extrai a base de leads inteira de todas as contas (dados pessoais: nome, telefone,
e-mail, cidade, historico de etapa) – vazamento de LGPD em massa. Nao foi possivel confirmar o estado real em
producao porque a view nao esta versionada; por isso o achado esta em media e nao em critica.

## Condicao de explorabilidade

Explorabilidade depende da configuracao real da view em producao (security_invoker e GRANT), nao verificavel a
partir do repositorio.

## Correcao sugerida

Primeiro, verificar em producao: SELECT relname, reloptions FROM pg_class WHERE relname = 'vw_relatorio_leads'.
Em seguida versionar a view numa migration com WITH (security_invoker = true) e um GRANT explicito, e mover o
export para o backend, atras de authMiddleware + verificacao de papel + filtro por aces_id (o padrao de
/api/collections/imports em api-server.ts:1114-1128). Remover a condicao acesId === 535 do frontend, que e
regra de negocio hardcoded e nao controle de acesso.

## Criterios de aceite

- [ ] vw_relatorio_leads definida em supabase/migrations/ com security_invoker = true.
- [ ] Consulta a vw_relatorio_leads com JWT de um tenant retorna apenas linhas daquele aces_id.
- [ ] Export passa por endpoint do backend com verificacao de papel; nenhuma chamada direta do frontend a view.
- [ ] A constante 535 nao aparece mais como criterio de acesso no frontend.

--- FIM ISSUE 8 ---
```

ISSUE 9 — [Seguranca] Segredo default publico no docker-compose e ausencia de validacao de startup

```
--- ISSUE 9 ---

## Titulo

[Seguranca] Segredo default publico no docker-compose e ausencia de validacao de startup

## Labels

`security`, `severity:medium`

## Severidade

MEDIA (achado F09 da auditoria de 04 de setembro de 2026)

## Problema

O valor 'local-only-rb-webhook-secret' esta versionado e vira segredo real de assinatura sempre que
RB_WEBHOOK_JWT_SECRET nao for exportado no shell que sobe o compose. Esse segredo assina os JWT HS256 de
webhook do RB (rb-connection-service.ts:162-183), portanto quem conhece o default forja tokens offline para
qualquer connection_id/aces_id, sem nem precisar do /webhook/login. Nao existe validacao de startup que
rejeite valores default ou de baixa entropia: requireEnv apenas verifica se a string e nao vazia. Os
placeholders 'local-dev-verify-token' e 'local-dev-app-secret' de .env.local.example:57-58 tem o mesmo
problema de classe. Nota positiva: o stack de producao (docker-stack.backend.yml) usa ${VAR} puro, sem
default, em todos os 18 segredos.

## Evidencia

- `docker-compose.yml:25`
```

```

- `Project/IA/api-server.ts:274-280`
- `.env.local.example:57-58`

```ts
docker-compose.yml:25
- RB_WEBHOOK_JWT_SECRET=${RB_WEBHOOK_JWT_SECRET:-local-only-rb-webhook-secret}

// api-server.ts:274 – requireEnv so checa presenca, nao qualidade
function requireEnv(name: string) {
 const value = process.env[name];
 if (!value) {
 throw new Error(`Variavel de ambiente obrigatoria ausente: ${name}`);
 }
 return value;
}
```

## Impacto

Falsificacao completa dos tokens de webhook do RB em qualquer ambiente que herde o default, combinando com F01 para acesso cross-tenant as operacoes de visagismo. Em compose de dev/homolog com dados reais, o impacto e o mesmo de producao.

## Condicao de explorabilidade

Explorável apenas em ambientes que subiram sem exportar a variavel. O stack de producao nao tem defaults.

## Correcao sugerida

Remover o default do compose (usar ${RB_WEBHOOK_JWT_SECRET:?RB_WEBHOOK_JWT_SECRET obrigatorio}, que faz o compose falhar cedo) e adicionar validacao de startup no backend: uma funcao requireStrongSecret(name) que rejeite valores de uma denylist de defaults conhecidos e exija comprimento minimo (>= 32 bytes) quando NODE_ENV=production. Rotacionar o segredo em todos os ambientes que possam ter subido com o default.

## Criterios de aceite

- [ ] Nenhum ${VAR:-valor} com valor de segredo nos arquivos compose/stack (apenas flags e nomes de modelo).
- [ ] Backend aborta o startup em producao se algum segredo for default conhecido ou tiver menos de 32 bytes.
- [ ] Teste unitario da validacao de startup cobrindo default conhecido, segredo curto e segredo valido.
- [ ] RB_WEBHOOK_JWT_SECRET rotacionado em todos os ambientes.

--- FIM ISSUE 9 ---

```

ISSUE 10 — [Seguranca] Segredos de terceiros commitados no historico do Git

```

--- ISSUE 10 ---

## Titulo

[Seguranca] Segredos de terceiros commitados no historico do Git

## Labels

`security`, `severity:medium`

## Severidade

MEDIA (achado F10 da auditoria de 04 de setembro de 2026)

## Problema

Os arquivos foram removidos do HEAD (nao aparecem em 'git ls-tree -r HEAD'), mas continuam recuperaveis em qualquer clone via 'git show <commit>:<path>'. Verificado que nenhum outro arquivo versionado contem segredo (varredura por AizaSy*, sk-*, JWT HS256 e blocos PRIVATE KEY em todo o 'git ls-files' voltou vazia) e que todos os .env* estao corretamente ignorados. Os dois valores sao de risco baixo em si – o JWT do RB tem exp em 2026-07-08 (expirado) e a chave Supabase e uma anon key (publica por design) de um projeto de terceiro. O problema real e de processo: segredos chegaram ao historico e nao houve rotacao nem limpeza.

## Evidencia

- `referencias/Cobranca API/Curl.txt (commit 02333eb e seguintes)`
- `referencias/eMetizaun/CONFIGURACAO_SUPABASE.md (commit 02333eb e seguintes)`

```

```

``ts
# git show 02333eb:"referencias/Cobranca API/Curl.txt"
#   -> Bearer JWT da API RB. Payload decodificado:
#     {"iss":"<id do emissor redigido>", "exp":"2026-07-08T14:42:19", ...,
#     "secrets":"eyJzZWNyZXRzIjoi..."}

# git show 02333eb:referencias/eMetizaun/CONFIGURACAO_SUPABASE.md
#   -> chave Supabase de OUTRO projeto. Payload decodificado:
#     {"iss":"supabase", "ref":"hkqrgomafbohittsdnea", "role":"anon", ...}
...

## Impacto

Direto: baixo – token RB expirado e anon key publica de outro projeto. Indireto: o campo 'secrets' embutido no
JWT do RB pode conter material reutilizavel, e o padrao demonstra ausencia de barreira de commit para
segredos. O mesmo caminho poderia ter levado uma service_role ao historico.

## Condiçao de explorabilidade

Requer acesso de leitura ao repositorio (clone). Ambos os valores tem impacto direto limitado.

## Correcao sugerida

Confirmar com o parceiro RB que o token esta revogado (nao apenas expirado). Adotar varredura de segredos no
pre-commit e na CI (gitleaks ou equivalente). Avaliar a limpeza do historico com git filter-repo para os dois
caminhos, coordenando o rewrite com quem tem clones. Manter a regra de nunca versionar diretorios de
referencia com material de integracao.

## Criterios de aceite

- [ ] Token RB confirmado como revogado pelo parceiro.
- [ ] Varredura de segredos rodando no pre-commit e na CI, com falha bloqueante.
- [ ] Decisao registrada sobre limpeza do historico (executada ou aceita como risco, com justificativa).
- [ ] gitleaks detect no repositorio nao retorna achados novos.

--- FIM ISSUE 10 ---

```

ISSUE 11 — [Seguranca] Claims de papel e tenant no JWT ficam obsoletas ate a renovacao do token

```

--- ISSUE 11 ---

## Titulo

[Seguranca] Claims de papel e tenant no JWT ficam obsoletas ate a renovacao do token

## Labels

`security`, `severity:low`

## Severidade

BAIXA (achado F17 da auditoria de 04 de setembro de 2026)

## Problema

A RLS decide papel e tenant a partir das claims do JWT, gravadas em app_metadata pelo trigger
sync_aces_id_to_jwt. app_metadata nao e editavel pelo usuario (so pelo service_role), portanto nao ha spoofing
– esse ponto esta correto. O efeito colateral e temporal: rebaixar um usuario de ADMIN para VENDEDOR via
PATCH /api/admin/users/:id/role atualiza a tabela e o app_metadata, mas o access token em uso continua
carregando crm_role = ADMIN ate expirar (1h por default no Supabase). Durante essa janela, escritas diretas do
frontend pela chave anon ainda passam pelas policies de ADMIN. O backend nao sofre disso porque
authenticate() (sdr-agent-gemini.ts:2647) le o papel ao vivo de crm.users. AuthContext.tsx:76-92 detecta a
divergencia e chama refreshSession(), o que mitiga o caso comum, mas e logica de cliente e pode ser evitada.

## Evidencia

- `supabase/migrations/20260411161000_move_crm_context_to_jwt.sql:3-51`
- `src/contexts/AuthContext.tsx:76-92`
- `Project/IA/sdr-agent-gemini.ts:2647-2679`

``ts

```

```
-- 20260411161000:43 — RLS le o papel do app_metadata do JWT
CREATE OR REPLACE FUNCTION public.current_crm_role()
RETURNS crm.user_role LANGUAGE sql STABLE SECURITY DEFINER
AS $$ SELECT NULLIF(auth.jwt() -> 'app_metadata' -> 'crm_role', '')::crm.user_role; $$;

-- o trigger sync_aces_id_to_jwt atualiza auth.users.raw_app_meta_data,
-- porem o JWT ja emitido continua valido com o papel antigo
```


Impacto

Janela de privilegio residual de ate uma expiracao de token depois de um rebaixamento de papel ou troca de tenant, aplicavel somente as escritas que o frontend faz direto no Postgres (automation_*, pipelines, pipeline_stages, empresas, calendar.*).

Condicao de explorabilidade

Janela limitada ao tempo de vida do access token e parcialmente mitigada pelo refresh automatico do AuthContext.

Correcao sugerida

Nas policias mais sensiveis, resolver o papel na tabela em vez da claim (por exemplo, uma funcao SECURITY DEFINER que faca SELECT role FROM crm.users WHERE auth_user_id = auth.uid()), aceitando o custo de um lookup. Alternativamente, invalidar as sessoes do usuario no servidor ao alterar papel (auth.admin.signOut do usuario) e reduzir o tempo de vida do access token.

Criterios de aceite

- [] Rebaixar um usuario de ADMIN para VENDEDOR revoga imediatamente as escritas admin feitas direto pelo frontend.
- [] Teste de integracao: alterar papel e, sem renovar o token, confirmar que a escrita admin e negada.
- [] Decisao documentada entre lookup na tabela e invalidacao de sessao.

--- FIM ISSUE 11 ---


```

## ISSUE 12 — [Seguranca] Reflexao e redirecionamento sem validacao na borda HTTP (challenge de webhook e returnPath do OAuth)

```
--- ISSUE 12 ---
```

### ## Titulo

[Seguranca] Reflexao e redirecionamento sem validacao na borda HTTP (challenge de webhook e returnPath do OAuth)

### ## Labels

`security`, `severity:low`

### ## Severidade

BAIXA (agrupa os achados F11, F12 da auditoria de 04 de setembro de 2026)

### ## Problema

Dois pontos da borda HTTP devolvem valor controlado pelo requisitante sem validacao adequada.

**\*\*1) XSS refletido no challenge de verificacao (F11).** Confirmado empiricamente com o Express 5.2.1 instalado no projeto: `res.send(string)` define `Content-Type: text/html; charset=utf-8`. O valor de `hub.challenge` vem direto da query string e e devolvido sem escape, sem `Content-Type` explicito e sem `X-Content-Type-Options`. `GET /api/webhook/instagram?hub.mode=subscribe&hub.verify_token=<token>&hub.challenge=<script>... executa script na origem do backend`. A exploracao depende de conhecer `META_WEBHOOK_VERIFY_TOKEN` ou `INSTAGRAM_WEBHOOK_VERIFY_TOKEN` — valores que `.env.local.example:57` sugere como `'local-dev-verify-token'`, do tipo que costuma sobreviver a homologacao.

**\*\*2) Open redirect por URL protocolo-relativa (F12).** Verificado em Node: a string `'//evil.com'` satisfaz o regex de sanitizacao (inicia com `'/'` e todos os caracteres seguintes estao na classe permitida) e, resolvida com new URL contra a base, produz uma URL de outra origem, porque `'/'` e interpretado como URL protocolo-relativa. O valor sai de `beginOAuth` (`req.body.returnPath`, `api-server.ts:3810`), e persistido no state assinado e usado no `res.redirect(303, ...)` do callback.

### ## Evidencia

```

- `Project/IA/api-server.ts:757-767`
- `Project/IA/api-server.ts:784-806`
- `Project/IA/meta-webhook.ts:44-54`
- `Project/IA/instagram-service.ts:89-92`
- `Project/IA/instagram-service.ts:151-156`
- `Project/IA/api-server.ts:3822-3852`

```ts
// api-server.ts:757
app.get("/api/webhook/meta", (req, res) => {
  const challenge = metaWebhookProcessor.verifyChallenge(req.query as Record<string, unknown>);
  if (!challenge) { res.status(403).send("Forbidden"); return; }
  res.status(200).send(challenge); // Express: string -> text/html
});

// api-server.ts:805 – mesmo padrao com hub.challenge cru
res.status(200).send(challenge);

// -----

// instagram-service.ts:89
function sanitizeReturnPath(value: string | null | undefined) {
  const normalized = value?.trim() || "/admin?section=instances";
  return /^[A-Za-z0-9\_\?&=.%-]*$/.test(normalized) ? normalized : "/admin?section=instances";
}

// "//evil.com" passa no regex (comeca com / e usa apenas caracteres permitidos)
// instagram-service.ts:153
const url = new URL(sanitizeReturnPath(returnPath), `${base}/`);
// new URL("//evil.com", "https://crm.example.com/") === "https://evil.com/"
```

Impacto

F11: Execução de script na origem do backend. Como o frontend guarda a sessão em localStorage de outra origem, não há roubo direto de token; o impacto prático é phishing convincente hospedado no domínio da API e abuso de qualquer coisa ligada a essa origem.

F12: Redirecionamento para domínio externo a partir de uma URL do backend confiável. O returnPath vem do próprio usuário autenticado que inicia o fluxo, então o cenário principal é auto-redirecionamento; o valor real está em usar o domínio da API como trampolim em phishing e em burlar filtros que confiam no domínio de origem.

Condição de explorabilidade

F11: Requer conhecer o verify token do webhook (segredo compartilhado com a Meta).

F12: Requer que o próprio usuário inicie o fluxo OAuth com o returnPath manipulado.

Correção sugerida

F11: Responder com Content-Type explícito e não-HTML: res.type('text/plain').send(challenge), ou res.set('Content-Type', 'text/plain; charset=utf-8'). Validar que challenge começa com /^[A-Za-z0-9_-]{1,256}$/ antes de refletir e adicionar X-Content-Type-Options: nosniff (o endpoint de mídia do Instagram em api-server.ts:815 já faz isso e serve de referência).

F12: Rejeitar caminhos que comecem com '/' (ou com '\\'): exigir /^[\\/(?!\\)[A-Za-z0-9_\?&=.%-]*$/ e, como defesa em profundidade, validar depois do new URL que url.origin === new URL(base).origin, caindo no default quando divergir.

Critérios de aceitação

- [] Resposta dos dois endpoints GET de verificação usa Content-Type text/plain.
- [] challenge validado por allowlist de caracteres antes de ser refletido.
- [] Header X-Content-Type-Options: nosniff presente nas respostas.
- [] Teste de rota confirmando que payload com < > não é devolvido como HTML.
- [] sanitizeReturnPath devolve o default para '//evil.com', '///evil.com' e '\\evil.com'.
- [] buildFrontendRedirect confirma que a origem final é a do frontend configurado.
- [] Teste unitário com os vetores acima.

--- FIM ISSUE 12 ---

```

## ISSUE 13 — [Seguranca] Endurecimento da borda do backend: vazamento de erro do banco, limite de corpo de 150 MB e comparacao de segredo sem tempo constante

--- ISSUE 13 ---

### ## Titulo

[Seguranca] Endurecimento da borda do backend: vazamento de erro do banco, limite de corpo de 150 MB e comparacao de segredo sem tempo constante

### ## Labels

`security`, `severity:low`

### ## Severidade

BAIXA (agrupa os achados F13, F14, F15 da auditoria de 04 de setembro de 2026)

### ## Problema

Tres ajustes de defesa em profundidade no mesmo arquivo de entrada do backend.

**\*\*1) Erro cru do banco na resposta (F13).** \*\* `HttpError` guarda o terceiro argumento em `details` (`sdr-agent-gemini.ts:1122-1131`) e o handler global serializa esse campo na resposta. Em 19 pontos de `api-server.ts` o objeto passado e o erro cru do `supabase-js/PostgREST`, que carrega `message`, `details`, `hint` e `code` — muitas vezes com nome de tabela, coluna, constraint e fragmento de SQL.

**\*\*2) Limite de corpo JSON em 150 MB (F14).** \*\* O parser global aceita ate 150 MB por requisicao quando nenhuma das duas variaveis esta definida. Esse limite vale para todas as rotas montadas depois, inclusive os webhooks nao autenticados (`/webhook/evolution`, `/api/webhook/gupshup`). Cada corpo e materializado em memoria antes de qualquer verificacao de credencial. Rotas especificas fazem melhor: o webhook de `collections` usa `express.raw({ limit: '1mb' })` (`api-server.ts:553-557`) e `/webhook/image` tem guarda de 15 MB (`api-server.ts:559-567`).

**\*\*3) Comparacao de segredo fora de tempo constante (F15).** \*\* Os dois handlers comparam segredos com `===`, que faz curto-circuito no primeiro byte diferente. Os webhooks `Meta`, `Instagram` e `collections` fazem o correto, com `crypto.timingSafeEqual` (`meta-webhook.ts:67-70`, `instagram-webhook.ts:119`, `collections/webhook-security.ts:51`). Agravante no `Gupshup`: o segredo pode vir em `req.query.secret`, e query strings vazam para access logs, referrers e historico de proxy.

### ## Evidencia

```
- `Project/IA/api-server.ts:4315-4321`
- `Project/IA/sdr-agent-gemini.ts:1122-1131`
- `Project/IA/api-server.ts:2093 (e outras 18 ocorrencias do padrao)`
- `Project/IA/api-server.ts:568-580`
- `Project/IA/sdr-agent-gemini.ts:15834`
- `Project/IA/api-server.ts:705-709`

``ts
// api-server.ts:4315 — handler global devolve details verbatim
app.use((error: unknown, req: Request, res: Response, _next: NextFunction) => {
 if (error instanceof HttpError) {
 return res.status(error.statusCode).json({
 error: error.message,
 details: error.details ?? null, // <-- objeto de erro do PostgREST
 });
 }
});

// padrao dos call sites (19 ocorrencias):
// throw new HttpError(500, "Nao foi possivel carregar usuarios", error);

// -----

app.use(
 express.json({
 limit:
 process.env.JSON_BODY_LIMIT ?? process.env.WEBHOOK_JSON_LIMIT ?? "150mb",
 verify: (req, _res, buf) => { ... },
 }),
```

```
);

// -----

// sdr-agent-gemini.ts:15834 (Evolution)
return provided === this.config.evolutionWebhookSecret;

// api-server.ts:706 (Gupshup) – tambem aceita o segredo pela query string
const providedSecret =
 req.header("x-gupshup-secret") || req.header("x-webhook-secret") ||
 asString(req.query.secret);
if (configuredSecret && providedSecret !== configuredSecret) { throw new HttpError(401, ...); }
````  
  
## Impacto  
  
**F13:** Divulgacao de informacao: mapeamento do schema interno (tabelas, colunas, constraints, funcoes) para  
qualquer usuario autenticado, o que facilita a fase de reconhecimento de outros ataques. Sem impacto direto de  
integridade ou confidencialidade de dados.  
  
**F14:** Exaustao de memoria/CPU com poucas requisicoes concorrentes, levando a indisponibilidade do backend.  
Nao ha impacto de confidencialidade.  
  
**F15:** Canal lateral de tempo teoricamente explorável para recuperar o segredo byte a byte; na pratica,  
ruidoso sobre rede. O vazamento do segredo por query string em logs e o risco mais concreto.  
  
## Condicao de explorabilidade  
  
**F13:** Requer sessao autenticada capaz de provocar erro em alguma rota.  
  
**F14:** Nenhuma; alcançavel nas rotas de webhook sem autenticacao.  
  
**F15:** Nenhuma para o vazamento em log; o ataque de tempo exige muitas amostras.  
  
## Correcao sugerida  
  
**F13:** Registrar o erro completo no log do servidor e responder apenas com a mensagem tratada mais um  
requestId de correlacao. Se detalhes forem uteis ao cliente, expor uma lista fechada de campos seguros (por  
exemplo, apenas code mapeado para mensagem amigavel) e nunca o objeto cru – em producao, omitir details por  
completo.  
  
**F14:** Reduzir o default global para algo na ordem de 1-2 MB e elevar o limite apenas nas rotas que  
comprovadamente precisam de payload grande, montando um express.json dedicado por rota, como ja e feito em  
/webhook/image.  
  
**F15:** Comparar com crypto.timingSafeEqual sobre buffers, checando o tamanho antes. Remover req.query.secret  
como fonte aceita, mantendo apenas headers.  
  
## Criterios de aceite  
  
- [ ] Respostas de erro em producao nao contem message/details/hint do PostgREST.  
- [ ] Erro completo permanece no log do servidor, correlacionado por requestId devolvido ao cliente.  
- [ ] Nenhum call site passa objeto de erro cru para o construtor de HttpError sem sanitizacao.  
- [ ] Default global do express.json em <= 2mb.  
- [ ] Rotas que precisam de payload maior tem parser dedicado com limite documentado.  
- [ ] Teste confirmando 413 para corpo acima do limite global em rota comum.  
- [ ] Comparacao de segredo de webhook usa timingSafeEqual em todos os handlers.  
- [ ] Segredo da Gupshup aceito somente por header.  
- [ ] Teste unitario cobrindo segredo correto, incorreto e de tamanho diferente.  
  
--- FIM ISSUE 13 ---
```

ISSUE 14 — [Seguranca] Endurecimento de XSS no frontend: CSP, sanitizacao obrigatoria e sinks latentes de <style>

--- ISSUE 14 ---

Titulo

[Seguranca] Endurecimento de XSS no frontend: CSP, sanitizacao obrigatoria e sinks latentes de <style>

Labels

`security`, `severity:low`

Severidade

BAIXA (agrupa os achados F16, F18 da auditoria de 04 de setembro de 2026)

Problema

Nenhum XSS explorável foi encontrado no frontend, mas duas condicoes ampliam o impacto de um XSS futuro.

****1)** Sessao em localStorage (F16).****** Access token e refresh token ficam em localStorage, acessiveis a qualquer script da origem. A auditoria nao encontrou nenhum sink de XSS explorável no frontend (nao existe innerHTML, eval nem new Function em src/, e os dois dangerouslySetInnerHTML sao CSS estatico), portanto isto e amplificacao de risco, nao vulnerabilidade ativa. Registrado porque o projeto nao tem biblioteca de sanitizacao instalada, o que aumenta a chance de um sink aparecer em codigo futuro.

****2)** Sink latente de CSS em componente sem uso (F18).****** Sao os dois unicos usos de dangerouslySetInnerHTML no projeto. O de material-ui-dropdown-menu.tsx:198 e CSS 100% estatico, sem interpolacao – sem risco. O de chart.tsx:70 interpola id e cores num bloco <style>, mas hoje id vem de React.useId() (chart.tsx:40-41) e as cores vem de ChartConfig definido em codigo; confirmado que ChartContainer nao e usado em nenhum lugar fora de chart.tsx, ou seja, e boilerplate do shadcn/ui atualmente morto. Registrado como informativo porque viraria injecao de CSS no momento em que alguem alimentar o config com dado de banco (por exemplo, cor de etapa ou de tag escolhida pelo usuario).

Evidencia

- `src/integrations/supabase/client.ts:15-24`
- `src/components/ui/chart.tsx:61-88`
- `src/components/ui/material-ui-dropdown-menu.tsx:193-215`

```
``ts
export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY, {
  db: { schema: 'crm' },
  auth: {
    storage: localStorage,          // access + refresh token legiveis por JS
    persistSession: true,
    autoRefreshToken: true,
  }
});
```

// -----

```
// chart.tsx:70 – id e cores entram num <style> sem escape
<style>
  dangerouslySetInnerHTML={{
    __html: Object.entries(THEMES).map(([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig.map(([key, itemConfig]) => {
  const color = itemConfig.theme?.[theme] || itemConfig.color;
  return color ? `--color-${key}: ${color};` : null;
}).join("\n")}
}).join("\n"),
  }}
/>
...`
```

Impacto

****F16:**** Se um XSS surgir no frontend, o atacante extrai o refresh token e mantem acesso persistente a conta mesmo depois do fechamento da aba, com o tenant e o papel da vitima.

****F18:**** Nenhum hoje. Se alimentado por dado do usuario, permitiria injecao de CSS (exfiltracao por seletor de atributo, defacement); nao permite execucao de script em navegadores atuais dentro de <style>.

Condicao de explorabilidade

****F16:**** Sem impacto isolado; depende de um XSS futuro no frontend.

****F18:**** Nao explorável no estado atual do codigo: o componente nao e instanciado em nenhuma pagina.

Correcao sugerida

****F16:**** Ao menos adicionar Content-Security-Policy restritiva no index.html/headers da Vercel para reduzir a superficie, e incluir DOMPurify como dependencia obrigatoria antes de qualquer futura renderizacao de HTML. Migrar para armazenamento em cookie httpOnly exige um proxy de autenticacao no backend e deve ser avaliado como trabalho maior.

****F18:**** Se o componente voltar a ser usado, validar as cores por allowlist (por exemplo /^[0-9a-fA-F]{3,8}\$/ ou nomes de token conhecidos) e as chaves por /^[a-zA-Z0-9_-]+\$/ antes da interpolacao. Caso siga sem uso, remover chart.tsx reduz a superficie.

Criterios de aceite

- [] CSP restritiva publicada para o frontend (sem unsafe-inline em script-src).
- [] Decisao registrada sobre manter localStorage ou migrar para cookie httpOnly.
- [] Regra de lint bloqueando novos dangerouslySetInnerHTML sem sanitizacao.
- [] Componente removido, ou cores/chaves validadas por allowlist antes da interpolacao.
- [] Regra de lint impedindo novos dangerouslySetInnerHTML sem sanitizacao explicita.

--- FIM ISSUE 14 ---